Tail-latency amplification and coupled thread-pool saturation in cloud microservice call graphs, with deadline-aware overload control

Devarsh Patel[1] and Ranjithkumar S[1*]

[1]School of Computer Science and Engineering (SCOPE), Vellore Institute of Technology (VIT), Vellore, Tamil Nadu, India.

*Corresponding author(s). E-mail(s): ranjithkumar.s@vit.ac.in
E-mail: devarsh.patel2024@vitstudent.ac.in

Abstract

Microservice requests fan out across dependency graphs, and each synchronous upstream call holds a worker thread until its downstream calls return. We state a coupled $\boldsymbol{G/G/m}$ holding-time model for bounded call graphs and evaluate it on a process-per-vertex gRPC testbed with synchronous, asynchronous, and hybrid runtimes, open-loop Poisson load, and coordinated-omission-corrected latency. Across 540 completed runs in five phases, P99.9 latency at the highest stable load was 2.8 to 8.1 times the critical-path median sum for 17 of 18 call-graph shapes, and 20.5 times for a width-eight in-cast. Along chains, the saturation horizon fell from 234–312 to 57–77 requests per second as depth grew from two to eight, while the root pool reached utilization 0.75 to 0.79 with the host about 10% busy. Asynchronous and hybrid chains of depth eight did not saturate up to 191 requests per second. The model overpredicted stable-load P99.9 by about 1.6 times at the median, and the chain error traces to the root's fixed shape parameter. Two added diagnostics did not remove the fan-out error. Under capacity faults at high load, a deadline-budget controller (DAC-Shed) kept the P99.9 of successful requests below 260 ms, where a static circuit breaker and naive retries reached 2.7 s or more. Naive retries delivered more goodput under transient errors, and an offline replay of a budget-gated single retry recovered most of that goodput. Near saturation, distributed tracing with sampling rates of 0.25 or higher raised median P99 by 4.3 to 10.6 ms across all six configurations (pooled bootstrap median +5.5 ms, exact sign test $\boldsymbol{p = 0.03125}$), representing a conservative lower

bound given higher baseline background load. Relaxing the quiet-host gate to 1.2 cores across stored contaminated runs leaves all 18 Phase 1 saturation brackets and all Phase 3–5 controller rankings unchanged.

Keywords: Microservices, Service meshes, Tail latency, Queueing theory, Overload control, Cloud computing

1 Introduction

Cloud back ends are assembled from fine-grained remote procedure calls across dependency graphs [1, 2]. A request completes only when its slowest required branch completes, so a modest queueing delay at one shared downstream service can dominate the end-to-end tail [1]. Production traces show that these graphs are deep and skewed, with a few heavily shared services carrying most of the traffic [3], and the study of their faults has become a field of its own [4]. Cloud hosts are heterogeneous and shared, so the latency of one service depends on the work that runs beside it [5, 6]. Outages of large cloud services are frequent enough to be studied systematically [7]. Measurements of single-server tail latency attribute much of the tail to interference and scheduling rather than to the application itself [8, 9], and warehouse-scale profiling shows that these effects are pervasive [10].

Thread-pool concurrency adds a second source of coupling that is easy to overlook. A synchronous upstream handler holds its worker thread for the full duration of each downstream call. An upstream pool can therefore saturate while the host processors are still mostly idle, because the scarce resource is the thread, not the CPU. The pool then accumulates queueing delay that belongs to a downstream tier. Metastability studies describe how this kind of overload feedback keeps a system degraded after the original trigger has passed [11, 12]. Admission-side controllers address overload at a single service, such as priority-based shedding [13] or credit-based admission for microsecond-scale RPCs [14]. Neither accounts for the thread-holding that a deadline budget must cross hop by hop. Sidecar proxies add a further per-hop cost, which has been measured for production-style service meshes [15].

The problem this paper addresses is how these two couplings combine as call graphs grow. A chain of synchronous hops and a wide in-cast of shared leaves saturate at different loads and for different reasons, and the operator needs to know which resource binds first, how far the load can rise before tails explode, and whether the latency cost of observability and proxying is material at those loads. These questions require controlled experiments under open-loop load, because closed-loop generators understate saturation by throttling their own arrivals when servers slow down [16].

We make four contributions. The first is a coupled holding-time model for bounded call graphs (Section 3). It predicts the root pool utilization from the arrival rate and the longest holding path, and it yields estimators for tail

amplification and for a saturation horizon. The second is a process-per-vertex gRPC testbed with three concurrency runtimes, open-loop Poisson load with coordinated-omission correction, and a quiet-host protocol that rejects runs disturbed by background load (Section 4). The third is a campaign of 540 completed runs in five phases that tests the model on chains, scatter-gather trees, diamond in-casts, and two DeathStarBench-derived sub-graphs, and that measures the serialization and sidecar tax, the tracing overhead, and the behavior of three overload controllers under three injected faults (Sections 5 and 6). The fourth is DAC-Shed, a deadline-budget and latency-gradient controller that is compared with a static circuit breaker and with naive retries, together with an offline replay of a budget-gated single retry.

The main results are as follows. P99.9 latency at the highest stable load was 2.8 to 8.1 times the critical-path median sum for 17 of the 18 Phase 1 shapes, and it was 20.5 times for a width-eight in-cast whose shared leaves approach saturation. Along chains, the saturation horizon fell from about 312 requests per second at depth two to about 77 at depth eight, while the root pool reached utilization 0.75 to 0.79 and the host processes used about ten per cent of the eight cores. At depth eight, synchronous chains timed out on 11.7% of requests at 115 requests per second, while asynchronous and hybrid chains of the same depth showed no timeouts up to 191 requests per second. At 77 and 115 requests per second the three runtimes used similar CPU, and above that the synchronous CPU stops rising because its requests fail. The model over-predicted measured stable-load P99.9 by about 1.6 times at the median. The chain error traces to the root's fixed shape parameter, and two added diagnostics reduced the chain error but did not remove the fan-out error. Under a one-second hang or a four-fold slowdown of the third vertex at high load, DAC-Shed kept the P99.9 of successful requests at or below 256 ms, where the static breaker and naive retries reached several seconds. Naive retries delivered the most goodput under transient errors, and an offline replay of a budget-gated single retry recovered 69.2% of first-attempt errors. Tracing with sampling rates of 0.25 or higher raised median P99 by 4.3 to 10.6 ms at high load (pooled median +5.50 ms, $p = 0.03125$); because the rate-zero baseline ran under higher background CPU (0.93 vs. 0.11–0.41 cores), this shift provides a conservative lower bound on high-load tracing overhead.

Section 8 discusses the scope and validity boundaries of our single-host process-per-vertex testbed, bytes-passthrough sidecar emulation, and trace-replay counterfactual. A multi-phase sensitivity analysis of the quiet-host gate (Section 7) confirms that relaxing the acceptance threshold from 1.0 to 1.2 cores leaves all 18 Phase 1 saturation brackets and all Phase 3–5 conclusions unchanged.

The remainder of the paper is organized as follows. Section 2 places the work against the literature. Section 3 states the model and the estimators, and Section 4 describes the testbed, the runtimes, the controllers, and the measurement protocol. Section 5 describes the campaign design and the quiet-host discards. Section 6 reports the results for the five research questions and

for the model diagnostics. Section 7 tests the sensitivity of the results to the quiet-host gate. Section 8 discusses the threats to validity, Section 9 gives deployment guidance, and Section 10 concludes.

The research questions that organize the evaluation are these. RQ1 asks how far high-quantile latency amplifies relative to the critical-path median, and whether the amplification depends on depth. RQ2 asks where saturation occurs, in the thread pool or on the processors, and how the three concurrency runtimes differ. RQ3 asks how large the serialization and sidecar costs are relative to CPU and latency. RQ4 asks how much distributed tracing adds to tail latency near saturation. RQ5 asks whether deadline-budget shedding protects successful requests under injected faults, and how it compares with a static breaker and with naive retries.

2 Related Work and Taxonomy

Table 1 summarizes twenty peer-reviewed systems and performance-modeling papers from ACM, USENIX, and other peer-reviewed venues. The selection is a targeted search and not a PRISMA-2020 systematic review, so it does not claim exhaustive coverage. The table records each work's scope, method, and the limitation that matters for multi-tier thread-pool analysis. This section groups the same works into four threads and states what each one leaves open.

2.1 Tail-latency amplification in microservice call graphs

Dean and Barroso [1] showed that parallel fan-out amplifies tail latency through order statistics, because a parent request completes only when its slowest branch returns. Their argument is qualitative and does not model how a branch's queueing delay propagates through the holding time of its parent. Gan et al. [2] introduced DeathStarBench, an end-to-end benchmark of microservice applications, and reported that RPC paths in these applications have substantially higher tail-to-median ratios than monolithic workloads. The benchmark is the source of the two sub-graphs used in our Phase 1 and Phase 3 campaign. Sriraman and Wenisch [17] provide a complementary microservice benchmark suite that reports tail latency for its services. Redundant requests address the same tail from the other side: cloning reduces stragglers at the cost of extra work [18], and the tail-at-scale argument favors hedged reads for idempotent calls [1]. Tail-latency guarantees for shared resources are studied by PriorityMeister [19] and SNC-Meister [20], which configure priorities and admission against tail SLOs in shared storage and networks rather than in microservice graphs.

Luo et al. [3] characterized microservice dependencies and performance in production traces from Alibaba clusters. Their analysis finds that a small number of heavily shared downstream services dominate the dependency graphs, which is the structure behind the diamond in-cast graphs we study. Zhou et al. [4] studied fault analysis and debugging of microservice systems through an

industrial survey, a benchmark system, and an empirical study. Their focus is fault diagnosis rather than the latency that builds up before a fault is declared.

The learning-based line of work uses historical traces to act on microservice performance. Seer [21], Sage [22], Sinan [23], and FIRM [24] diagnose QoS violations or adjust resource allocations in a closed loop. These systems treat each tier as a black-box compute node. None of them isolates how synchronous thread holding couples the saturation of an upstream pool to queueing at a downstream tier that is still below its CPU limit. That coupling is the subject of the present model. Cluster schedulers that account for interference and QoS act on the same problem at the placement level, as in Paragon [6] and Quasar [25], but they do not model the holding of threads along a call path.

2.2 Concurrency runtimes, thread-pool saturation, and metastability

When an upstream handler blocks a worker thread while it awaits downstream responses, queueing delay at a leaf tier directly inflates the upstream holding time. Bronson et al. [11] and Huang et al. [12] described metastable failures, in which overload feedback keeps a system degraded after the trigger has passed. Both works describe how retries and load feedback can hold a system in a degraded state, but they do not measure how that feedback depends on the depth and width of a call graph.

Staged event-driven designs made the thread pool itself a control point for overload [26]. Recent microsecond-scale schedulers preempt or steer requests to contain tails [27, 28], and they confirm that scheduling rather than raw capacity often sets the tail. The measurement methodology of this paper rests on a result about load generation. Schroeder et al. [16] showed that closed-loop generators understate the saturation knee of a server, because they throttle arrivals whenever the server slows down. Open-loop schedules with coordinated-omission correction avoid this bias, and our generator measures each request's latency from its intended send time. The statistical tools used to locate the saturation horizon and to fit tails are classical. Whitt [29] gives approximations for $GI/G/m$ stations that underlie the waiting-time term of our model. Kneedle [30] locates the knee of a curve from its curvature, bootstrap resampling [31] gives interval estimates without distributional assumptions, and peaks-over-threshold extreme value theory [32, 33] gives the generalized Pareto model for excesses above a high threshold. To our knowledge these tools have not been combined to test a multi-tier holding-time recursion across synchronous, asynchronous, and hybrid runtimes.

2.3 Service mesh sidecars, serialization, and observability overheads

Service meshes interpose user-space proxies and telemetry hooks on every RPC edge. Karandikar et al. [34] designed a hardware accelerator for Protocol Buffers after finding that serialization is a large share of cost in datacenter

workloads. Verma et al. [35] documented resource interference and scheduling behavior in Google's Borg cluster manager, which is the environment in which such sidecars are deployed. Zhu et al. [15] dissected the overheads of service mesh sidecars, and that study is the closest published work to the sidecar component of our Phase 3 experiment. Distributed tracing systems address causal context propagation. Pivot Tracing [36] propagates baggage along request paths, and Canopy [37] describes tracing at production scale.

Controlled experiments that separate per-hop serialization from sidecar socket forwarding across call-graph depths are scarce. The same holds for experiments that bound how span sampling inflates the tail near thread-pool saturation. Phase 3 and Phase 4 of this study address those two gaps directly, with an emulated sidecar and an instrumented chain, and they should be read as mechanism-level measurements rather than as production figures.

2.4 Overload control, circuit breaking, and deadline-aware shedding

Admission control and circuit breaking are the standard defenses against overload collapse. DAGOR [13] sheds requests by priority, keyed on queueing delay at each service, and it was deployed for WeChat's microservices. Breakwater [14] uses credit-based server-side admission for microsecond-scale RPCs. Both controllers protect a single tier. A local controller that ignores the remaining end-to-end budget can admit a request at an upstream hop that will expire before its critical path completes. Static circuit breakers and naive retries can also amplify a capacity fault, because retries add load to a tier that is already saturated. DAC-Shed combines hop-by-hop deadline-budget shedding with a latency-gradient concurrency limit adapted from TCP Vegas [38]. The gradient rule compares each 50 ms window with the smallest of the last twenty windows, which is the delay-based signal that Vegas uses for congestion. Scheduling that favors short work under overload has a long history [39], and the controller here applies a delay signal to admission rather than to request order.

2.5 Positioning

The four threads share a gap. Trace studies characterize the graphs, learning systems act on them, and admission controllers protect single tiers. None of the verified works, within the search scope above, measures coupled thread holding across a bounded call graph under controlled open-loop load with coordinated-omission correction, and none evaluates a deadline-budget controller against a static breaker and naive retries under the same injected faults. The contributions in Section 1 are aimed at that gap.

Table 1 Curated taxonomy of 20 peer-reviewed systems papers (ACM, USENIX, and other peer-reviewed venues). Every record was checked against Crossref or the USENIX presentation page. The selection is targeted, not a PRISMA-2020 review, and the limitations describe scope rather than quality.

Work	Venue	Scope	Method	Limitation for multi-tier thread-pool analysis
Dean and Barroso [1]	CACM 2013	Fan-out request paths	Tail-tolerant techniques	Tail amplification argued qualitatively; no coupled multi-tier queueing model
Gan et al. DeathStarBench [2]	ASPLOS 2019	End-to-end microservice applications	Hardware and software characterization	Characterizes applications; does not isolate the concurrency runtime or DAG depth
Gan et al. Seer [21]	ASPLOS 2019	Cloud microservice traces	Learned QoS-violation prediction	Prediction-oriented; no analysis of thread holding times
Gan et al. Sage [22]	ASPLOS 2021	Microservice dependency graphs	ML culprit localization	Diagnosis after the fact; no admission control
Zhang et al. Sinan [23]	ASPLOS 2021	Microservice resource allocation	ML-predicted end-to-end tail latency	Allocation-level control; thread-pool saturation below CPU capacity not modeled
Qiu et al. FIRM [24]	OSDI 2020	Multi-tier microservices	Learned SLO-driven resource reallocation	Container-level reallocation; no deadline-aware shedding
Luo et al. [3]	SoCC 2021	Production microservice traces	Dependency and runtime characterization	Observational; no controlled concurrency-model experiments
Zhou et al. [4]	IEEE TSE 2021	Microservice fault analysis	Industrial survey and benchmark system	Focus on fault debugging rather than latency saturation
Zhou et al. DAGOR [13]	SoCC 2018	WeChat microservices	Priority-based overload control	Evaluated on one production system; no DAG-coupled analysis
Huang et al. [12]	OSDI 2022	Incident reports across organizations	Metastable failure taxonomy	Qualitative incident study; no controlled latency measurements
Bronson et al. [11]	HotOS 2021	Distributed systems	Metastable failure model	Conceptual; not evaluated on DAG topologies
Cho et al. Breakwater [14]	OSDI 2020	Microsecond-scale RPC servers	Credit-based server-side overload control	Server-side admission; multi-tier coupling is not the focus
Brakmo et al. TCP Vegas [38]	SIGCOMM 1994	TCP congestion control	RTT-based delay-gradient control	Transport-layer method; the gradient controller here is adapted from it
Satopää et al. Kneedle [30]	ICDCS-W 2011	System behavior curves	Knee-point detection	General detector; no bootstrap intervals
Schroeder et al. [16]	NSDI 2006	Workload generation	Open versus closed system models	Establishes the open-loop requirement; not microservice-specific
Karandikar et al. ProtoAcc [34]	MICRO 2021	Protocol Buffers (de)serialization	Hardware accelerator	Hardware offload; general-purpose CPU serialization tax not characterized here
Verma et al. Borg [35]	EuroSys 2015	Cluster management	Large-scale scheduling	Scheduling, not request-path latency
Mace et al. Pivot Tracing [36]	SOSP 2015	Distributed monitoring	Dynamic causal monitoring	Monitoring design; tail-percentile observer effect not measured
Kaldor et al. Canopy [37]	SOSP 2017	Production tracing	End-to-end performance tracing	Tracing infrastructure; not a controlled observer-effect study
Zhu et al. [15]	SoCC 2023	Service mesh sidecars	Overhead decomposition	Sidecar overhead in production-style proxies; not DAG-coupled queueing

3 Coupled holding-time model

3.1 Call graphs and visit counts

Let $G = (V, E)$ be a bounded directed acyclic call graph with a single root v_0. The root serves external requests arriving as a Poisson process of rate λ_0. A vertex v is a $G/G/m_v$ station with m_v worker threads, a local service time of mean s_v, and a thread that stays occupied until its slowest child responds. Every root request makes $\nu(v)$ calls to vertex v, where $\nu(v_0) = 1$ and $\nu(v) = \sum_{u:(u,v)\in E} \nu(u)$. This recursion counts the root-to-v paths, so a vertex reached along several paths receives several calls per root request. The arrival rate at v is therefore $\lambda_v = \lambda_0\, \nu(v)$.

The four graph families in the campaign follow from this structure. A chain of depth d has d vertices in series, so every vertex has $\nu(v) = 1$. A scatter-gather tree of depth D has D levels, and every internal vertex has F children. Its total size is capped at 32 vertices, which bounds the F^D growth. A diamond in-cast of depth D and width W has W parallel branches of $D - 2$ middle vertices each, between the root and two shared leaves, so $|V| = 1+W(D-2)+2$. Each shared leaf receives W calls per root request, and the in-cast's visit counts therefore grow with width. The two DeathStarBench-derived sub-graphs have eight and twelve vertices.

3.2 Holding time, pool utilization, and waiting time

With per-hop network cost δ, the holding time of vertex v is the local service time plus the slowest child response,

$$S_v = s_v + \max_{u \in \mathcal{C}(v)} \big(\delta + R_u\big), \qquad S_v = s_v \text{ for leaves}, \tag{1}$$

where $\mathcal{C}(v)$ is the set of children of v and R_u is the response time of child u. The pool utilization of v is

$$\rho_v = \frac{\lambda_v S_v}{m_v}. \tag{2}$$

The waiting time uses the Sakasegawa-type approximation for $G/G/m$ stations [29],

$$W_v \approx \frac{c_a^2 + c_s^2}{2} \, \frac{\rho_v^{\sqrt{2(m_v+1)}-1}}{m_v(1-\rho_v)} \, S_v, \tag{3}$$

and the response time of the vertex is $R_v = W_v + S_v$. The factor $(c_a^2 + c_s^2)/2$ and the utilization exponent follow the heavy-traffic analysis of a single server [40], extended to m servers by Sakasegawa [41]. The model couples tiers through S_v, not through the arrival process: a deep downstream queue lengthens the holding time of every upstream thread, and that lengthening raises the upstream utilization in Eq. (2).

The prediction for the root is the response time R_{v_0}, with lognormal quantiles matched to its mean. The lognormal shape is set by the local service-time squared coefficient of variation, $c_s^2 = e^{\sigma^2} - 1$, and the same value enters the waiting-time term at every vertex. Path variance is not propagated, and downstream arrivals reuse the root's arrival SCV. Both are stated simplifications, and Section 3.5 tests the first of them directly. The model uses the campaign's parameters: $m_v = 8$ worker threads, $\delta = 0.5\,\text{ms}$ for the loopback hop, and $\sigma = 0.5$ for the local service time with a mean of 10 ms.

Little's law [42] links the quantities that the testbed records. The mean number of threads held at v equals the arrival rate times the mean holding time, $\lambda_v S_v$, and Eq. (2) is this quantity divided by the pool size. The measured pool utilization in Section 6 is computed from the recorded holding times in the same way.

3.3 Predictions that the experiments can falsify

The recursion makes two predictions. First, the root's pool utilization rises with the arrival rate and with the longest holding path, so deep chains saturate at lower rates. Second, a diamond in-cast vertex saturates when $\lambda_0 \nu(v) S_v = m_v$. Its visit count $\nu(v)$ grows with the number of converging branches, so a wider in-cast fills at a lower root rate. Section 6 tests both predictions against the measured tails and horizons, and it reports where they fail.

3.4 Estimators

Three estimators summarize each shape. The first two are amplification factors that compare a high quantile of end-to-end latency with a structural or load baseline,

$$\Gamma_{\mathrm{crit}}(\lambda,\alpha)=\frac{Q_\alpha\big(T_{\mathrm{req}}(\lambda)\big)}{\max_{p\in\mathrm{Paths}(G)}\sum_{v\in p}Q_{0.5}\big(T^0_{\mathrm{proc}}(v)\big)}, \tag{4}$$

$$\Gamma_{\mathrm{load}}(\lambda,\alpha)=\frac{Q_\alpha\big(T_{\mathrm{req}}(\lambda)\big)}{Q_\alpha\big(T_{\mathrm{req}}(\lambda_{\mathrm{base}})\big)}. \tag{5}$$

Here $\alpha = 0.999$, λ is the highest stable load level of the shape, and λ_{base} is the lowest level. The denominator of Γ_{crit} uses the baseline medians of each vertex's service time, summed along the longest path. The numerator is the measured P99.9 at the highest stable level.

The saturation horizon Θ^* is the load at which the curvature of a penalized cubic spline fitted to $\log_{10}$ of the P99.9 latency is largest. The spline minimizes $\sum_i (y_i - g(x_i))^2 + \eta \int (g'')^2$, with the smoothing parameter η chosen by generalized cross-validation. The curvature is $\kappa(\lambda) = |g''(\lambda)|/(1+g'(\lambda)^2)^{3/2}$, evaluated on a 2,000-point grid. A campaign shape has five load levels, so the knee can be located only to within one level spacing. A percentile interval on Θ^* therefore collapses to a single value, and we report two quantities instead. The bracket is the pair of measured load levels that enclose Θ^*. The stability is the fraction of 1,000 two-stage bootstrap resamples that return the same bracket. Each resample first draws the replicate runs at every level and then draws the requests within each run. No hypothesis test is used.

The bracket and stability reporting was adopted after the campaign data were collected, when percentile intervals proved degenerate. The point estimator is unchanged by that choice. Two cases need care. For synchronous chains of depth eight, the curvature peak lies at the first load level with timeouts, so the upper end of the bracket lies in saturated territory. For asynchronous and hybrid chains of depth eight, the curve is flat over the tested range and no knee occurs. The runtime comparison in Section 6.4 therefore uses the same offered loads rather than these brackets.

The tail shape is estimated by a generalized Pareto fit to the excesses above the 95th percentile of the measured latencies [32, 33]. The fit maximizes the likelihood with the location fixed at zero. Probability-weighted moments [43] are the usual alternative for small samples, and they were not used here. The excesses are pooled over the replicates at the highest stable level. For a threshold u with N_u excesses out of N samples, the extreme quantile is

$$\hat{Q}_\alpha = u + \frac{\hat{\sigma}}{\hat{\xi}}\left[\left(\frac{N}{N_u}(1-\alpha)\right)^{-\hat{\xi}} - 1\right]. \tag{6}$$

The 95% interval for $\hat{\xi}$ is a percentile bootstrap over 1,000 resamples, computed when at least 30 excesses are available. A load level is stable when its mean

root pool utilization is below 0.9 and its mean timeout fraction is at most 1%. Queues grow during an overloaded run, so the excesses of such a run are not stationary and are excluded from tail estimation.

3.5 Diagnostics for path variance and fan-out order statistics

The baseline recursion, called M0 here, takes the root's shape parameter from the single-vertex local SCV. It propagates neither the variance accumulated along a path nor the order-statistic shift of parallel branches. Two diagnostics test those omissions against the measurements. Both are computed after the campaign, on the same shape-level pairs, and neither replaces M0 as the primary model.

The first is a path-variance diagnostic. Along a serial chain of length L, the path-averaged SCV is $c^2_{s,\mathrm{path}} = (e^{\sigma^2} - 1)/L$, which gives the root shape parameter $\sigma^2_{\mathrm{root,path}} = \ln\bigl(1 + c^2_{s,\mathrm{path}}\bigr)$. The diagnostic tests whether M0's overprediction on deep chains comes from holding the root shape fixed at c^2_s.

The second is an order-statistic extension, called M1. Let $\mathcal{C}(v)$ be the children of v with $F = |\mathcal{C}(v)|$, and let child u arrive at $a_u = \delta + R_u$. Define the mean child variance $\bar{\sigma}^2_v = \frac{1}{F}\sum_{u\in\mathcal{C}(v)} \mathrm{Var}(R_u)$. Let α_F and β_F be the mean and variance of the maximum of F independent standard normal variates. The downstream stage that the parent thread waits for is then approximated by

$$\mathbb{E}[M_v] = \max_{u\in\mathcal{C}(v)} a_u + \alpha_F\,\bar{\sigma}_v, \qquad \mathrm{Var}(M_v) = \beta_F\,\bar{\sigma}^2_v, \tag{7}$$

and $S_v = s_v + \mathbb{E}[M_v]$ replaces the holding time in Eq. (3). The moments α_F and β_F are evaluated by numerical integration of the exact density of the maximum. For $F = 1$ the expressions reduce to the single-child case, with $\alpha_1 = 0$ and $\beta_1 = 1$. Unequal children use the same moments scaled by the root-mean-square child spread, which is an approximation. The waiting-time variance term that M1 feeds into Eq. (3) is heuristic, and we label it as such.

The M1 expressions are not a derivation of the variance of a holding time under a full stochastic model. They are an order-statistic correction that can be evaluated from the same inputs as M0. Section 6.1 reports what it does and does not change.

4 Testbed, runtimes, and controllers

4.1 Process-per-vertex architecture

Each vertex of a call graph runs as its own operating-system process with a gRPC server. The root vertex receives requests from an open-loop load generator running in a separate process. Edges are gRPC calls that carry the request payload and an absolute deadline in metadata. A run therefore involves one generator process, one process per vertex, and, in the sidecar phase, one

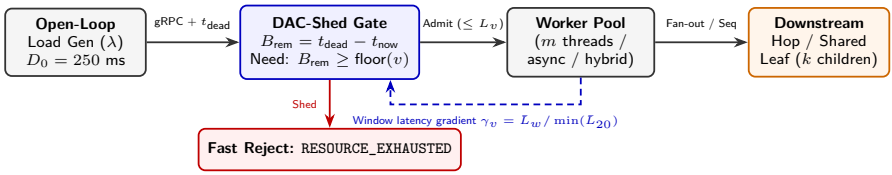


Fig. 1 Single-host gRPC microservice harness and DAC-Shed admission gate. Each RPC carries its absolute deadline t_{dead} in gRPC metadata. A vertex sheds a request before thread-pool queueing when the remaining budget is smaller than its critical-path floor, $B_{\text{rem}} < \text{floor}(v)$, or when in-flight work reaches the concurrency limit L_v. The latency gradient is computed over 50 ms windows and updates L_v.

proxy process per edge. Figure 1 shows the harness and the admission gate that DAC-Shed installs in front of each vertex's worker pool. Every process is pinned to the same eight-core host, so the generator, the vertices, and the proxies compete for the same processors. That shared host is the reason for the quiet-host protocol in Section 4.8.

4.2 Concurrency runtimes

Three runtimes are implemented, and they differ in where a thread is held. In the synchronous runtime, each vertex runs a bounded handler pool of eight worker threads. An in-flight request holds its handler thread across every downstream call, which is the situation the model describes. The asynchronous runtime uses `grpc.aio` with `uvloop`. A request never holds a thread across a downstream call, but its local work runs on the event loop, so the loop thread carries that work as well as protocol processing. The hybrid runtime keeps the asynchronous server and runs local work in a `ThreadPoolExecutor` with eight workers, so only the local processing time occupies the bounded pool. The downstream wait is performed on the loop and holds no thread.

The three runtimes therefore hold threads differently, and a fair comparison has to hold the offered load and the pool size fixed. The campaign does both. Phase 2 uses the same five offered loads for all three runtimes of each shape, so the curves can be compared point by point, and every runtime uses a pool of eight. The mechanism behind the differences is examined in Section 6.4.

4.3 Local work and service-time model

Local work is modeled as an I/O-bound operation. Each vertex draws a lognormal service time with a mean of 10 ms and a log-standard deviation of $\sigma = 0.5$, and 10% of the drawn time is spent in a busy loop, so the model contributes CPU as well as waiting. The model has no database or disk component. Sleeps overshoot on the test machine by about $0.3\,\text{ms} + 0.44\,t$ for a requested sleep t, and the implementation shortens each sleep by that fitted amount. The fit is the calibration step that keeps the realized service time close to the drawn one. Request payloads are 64 bytes in every phase except Phase 3, which sweeps payloads from 64 bytes to 64 KB.

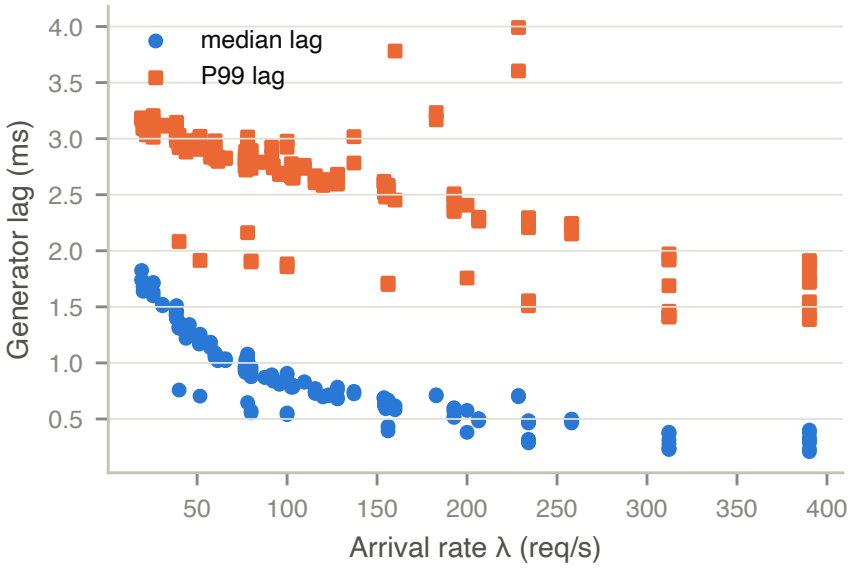


Fig. 2 Generator lag against offered rate, one point per run for all 180 Phase 1 runs. Circles show the median lag of a run and squares its P99 lag.

The analytical model uses the same parameters. The pool size is $m_v = 8$, the per-hop network cost is $\delta = 0.5$ ms, and the CPU cost per visit used for the CPU capacity bound is 1.5 ms.

4.4 Open-loop load generation and latency measurement

The load generator draws Poisson arrival offsets before each run starts. Each request has an intended send time, and its latency is measured from that intended time, not from the moment the request is actually sent. This is the coordinated-omission correction that open-loop measurement requires [16]. Generator lag, the difference between the intended and actual send times, is therefore added to the recorded latency, and it does not reduce the offered load.

Figure 2 shows the lag for all 180 Phase 1 runs. The median lag falls from about 1.8 ms at 19 req/s to 0.3 ms at 312 req/s, and the median per-run P99 lag stays between 1.6 and 3.8 ms across load levels. Lag therefore falls as load rises, which does not look like generator saturation. We attribute the residual lag to timer and scheduler jitter, which we did not measure separately. Each run lasts 90 s, and the first 10 s are excluded from the statistics. Each request is recorded with its outcome (success, error, or timeout), its latency, and its generator lag. Per-vertex hop records are kept for one request in ten. Every run also records process CPU counters and host CPU counters at a sampling interval of 0.5 s, together with a host-noise probe taken before the run.

4.5 Sidecar emulation and tracing

The sidecar in Phase 3 is a bytes-level pass-through process inserted in front of one vertex on each edge. It forwards the raw request and response unchanged, so it adds one hop and one byte copy, but it does not parse the protocol buffer. Its cost is measured from the caller's side, as the difference between the caller's wall time and the callee's own hop time, and from the proxy's own process CPU. The emulation is in-process and is not Envoy. The name "sidecar" refers to the cost structure of a proxy on the request path, not to a deployed mesh.

Tracing in Phase 4 creates a span for each hop with the `ExecuteHop` span name, propagates a W3C-style trace-parent key in the metadata, and exports the spans in one of two modes. The synchronous exporter writes each span on the request path. The batch exporter buffers spans and writes them in groups, which moves the export work off the request path. Section 6.6 reports whether the exporter mode changes the measured overhead. The sampling rate S selects the fraction of requests that are traced, and the rate-zero configuration is the baseline with no tracing at all.

4.6 Injected faults

Phase 5 injects three faults at one vertex, the third vertex of a four-vertex chain. The fault is active between 35% and 65% of each run, which is the middle 30%. A slowdown multiplies that vertex's service time by four. An error fault fails each request that the vertex receives with probability 0.3, and the failing response is returned to its caller as an injected error. A hang stalls each request at the vertex for one second. The injected-error status code was changed after the campaign, as described in Section 5.5, and the campaign results use the earlier code.

4.7 Overload controllers

Three controllers are compared in Phase 5. The static circuit breaker reproduces the static knobs of an Envoy-style proxy in-process. It caps the connections at 64 and the pending requests at 128, so at most 192 requests are in flight. It opens after five consecutive failures and rejects requests for two seconds while open. The breaker is an emulation of those knobs, not a deployment of Envoy, and its rejections count as failures for the success fraction.

The naive retry controller retries a failed call immediately, up to two attempts in total, and it has no retry budget. This is the policy that most client libraries use by default.

DAC-Shed has two parts, both of which are installed as a gRPC server interceptor. The first is deadline-budget shedding. Each request carries its absolute deadline, set to the root's 250 ms service-level budget and inherited along the call graph. A vertex rejects a request before admitting it when the remaining budget is smaller than its critical-path floor, the longest sum of baseline vertex costs from that vertex to a leaf, where each vertex's baseline cost is its mean service time plus the per-hop network cost. The floor is computed once per

Algorithm 1: DAC-Shed admission gate and latency-gradient concurrency controller

---

State at vertex v**:** critical-path floor $\text{floor}(v) = \max_{p:v\to\text{leaf}} \sum_{u\in p}(s_u + \delta)$, with s_u the mean service time and δ the per-hop cost; concurrency limit $L_v \in [L_{\min}, L_{\max}]$ with $L_{\min} = 4$, $L_{\max} = 128$, and initial value $L_v = 16$; decrease factor $\beta_{\text{dec}} = 0.85$; increase step $\alpha_{\text{inc}} = 1$; in-flight counter $I_v = 0$; a history $\mathcal{H}_{20}$ of the twenty most recent 50 ms window means; gradient $\gamma_v = 1.0$.

1. Upon RPC arrival at vertex v **with absolute deadline** t_{dead}**:**

1. Compute the remaining budget $B_{\text{rem}} \leftarrow t_{\text{dead}} - t_{\text{now}}$.
2. **if** $B_{\text{rem}} < \text{floor}(v)$ **then**
3. **return** `RESOURCE_EXHAUSTED` ▷ *Deadline-budget shed before pool wait*
4. **if** $I_v \geq L_v$ **then**
5. **return** `RESOURCE_EXHAUSTED` ▷ *Concurrency shed; does not change* L_v
6. $I_v \leftarrow I_v + 1$; execute local work and the downstream calls $u \in \mathcal{C}(v)$; $I_v \leftarrow I_v - 1$.
7. **if** the request succeeded **then** record its hop latency R_v in the current window w. Failed downstream calls are not recorded.

2. Every $\Delta t = 50$ **ms window tick at vertex** v**:**

8. Let $\bar{L}_w$ be the mean latency of the successful requests in window w, and $\bar{L}_{\min} = \min_{k\in\mathcal{H}_{20}} \bar{L}_k$.
9. Update the gradient: $\gamma_v \leftarrow \bar{L}_w / \bar{L}_{\min}$.
10. **if** $\gamma_v > 3.0$ **then** $L_v \leftarrow \max(L_{\min}, \lfloor \beta_{\text{dec}} L_v \rfloor)$.
11. **else if** $\gamma_v < 1.5$ **and** $I_v \geq 0.8\, L_v$ **then** $L_v \leftarrow \min(L_{\max}, L_v + \alpha_{\text{inc}})$.

3. Offline counterfactual only: a budget-and-congestion-gated single retry (`dacshed_retry`) is evaluated by trace replay in Section 6.7. It is not part of the controller that the campaign ran. Its gate permits one retry when the first attempt returned an injected error, when $(t_{\text{dead}} - t_{\text{now}}) \geq 2 \cdot \text{floor}(u)$, and when $\gamma_v < 3.0$.

Fig. 3 DAC-Shed admission gate and latency-gradient concurrency update. Constants are those of the implementation in `src/overload_controller.py`. The budget-gated retry in step 3 is evaluated offline on stored traces and is not part of the campaign controller.

vertex on the call graph, and it is passed to the vertex when the run is built. The second part is a concurrency limit that changes with a latency gradient, in the style of TCP Vegas [38]. Algorithm 1 in Fig. 3 gives the procedure.

Two properties of the controller are deliberate. Downstream failures do not move the concurrency limit, because a failing dependency is not evidence of local queueing, and shrinking the limit on it would shed healthy traffic. The controller's own rejections also never shrink the limit, so shedding cannot feed on its own signal. The controller is protected by a lock, because the synchronous runtime calls it from several handler threads.

4.8 Quiet-host protocol

Because the generator, the vertices, and the proxies share one host, background load is the main threat to a measurement. A background process that uses a

core during a run inflates the queueing of the vertices. Every run is therefore gated in two places. The start gate admits a run only when total host busy time is at most 25% of the eight cores, that is about two cores, for three consecutive one-second windows, and when the host-noise probe shows timer jitter of at most 3 ms. The start gate controls only when a run begins. The contamination test is applied during the run. It computes the background load as host busy time minus the CPU consumed by the harness's own processes, and it rejects any run whose mean background load exceeds 1.0 core.

A rejected run is archived in a folder with a `.contaminated` suffix and is repeated after a pause of 60 s. The archived folders keep their request, hop, and telemetry records, so the discarded attempts can be audited. Table 3 in Section 5 reports the discards by phase. The start gate and the contamination test do not use the same threshold, which is a design limitation discussed in Section 8. A run can start while background load is between 1.0 and 2.0 cores and then be discarded.

5 Campaign design and measurement record

5.1 Phases and factors

The campaign ran five phases on the eight-core host, in the order below. Each phase answers one research question, and each uses a factorial design over shape, runtime, load, replicate, and treatment factor. Table 2 lists the completed runs by shape and runtime.

- **Phase 1** covers all 18 shapes under the synchronous runtime, at five load levels with two replicates each. It gives $18 \times 5 \times 2 = 180$ runs. Phase 1 supplies the tail amplification, the saturation horizons, and the tail-shape estimates.
- **Phase 2** covers three shapes under the synchronous, asynchronous, and hybrid runtimes at five shared load levels with three replicates. It gives $3 \times 3 \times 5 \times 3 = 135$ runs.
- **Phase 3** sweeps call-graph depth (2, 4, and 8) against payload size (64 B, 1 KB, 4 KB, 16 KB, and 64 KB) with five replicates. The offered load is fixed at 30 req/s, and the sidecar emulation is enabled. It gives $3 \times 5 \times 5 = 75$ runs.
- **Phase 4** sweeps the tracing sampling rate over $\{0, 0.1, 0.25, 0.5, 1.0\}$ and the exporter mode (synchronous or batch) on a four-vertex chain at two loads, 0.2 and 0.6 of the chain's capacity. The baseline has six replicates per load, and each traced configuration has three. It gives $12 + 48 = 60$ runs.
- **Phase 5** crosses three injected faults with three controllers at five loads from 0.2 to 0.8 of the chain's capacity, with two replicates per cell. It gives $3 \times 3 \times 5 \times 2 = 90$ runs.

The five phases give $180 + 135 + 75 + 60 + 90 = 540$ completed runs, which matches the manifests. Every completed run has a manifest entry, a folder, and a status of `ok`.

Table 2 Campaign inventory: completed and planned runs per shape and runtime model.

Phase	Shape	Runtime	Completed/planned
1	chain_D2	sync	10/10
1	chain_D4	sync	10/10
1	chain_D6	sync	10/10
1	chain_D8	sync	10/10
1	deathstar_hotel_8	sync	10/10
1	deathstar_social_12	sync	10/10
1	diamond_D3_W4	sync	10/10
1	diamond_D3_W8	sync	10/10
1	diamond_D5_W4	sync	10/10
1	diamond_D5_W8	sync	10/10
1	diamond_D6_W4	sync	10/10
1	diamond_D7_W4	sync	10/10
1	sg_D2_F2	sync	10/10
1	sg_D2_F4	sync	10/10
1	sg_D2_F8	sync	10/10
1	sg_D3_F2	sync	10/10
1	sg_D3_F4	sync	10/10
1	sg_D4_F2	sync	10/10
2	chain_D8	aio	15/15
2	chain_D8	hybrid	15/15
2	chain_D8	sync	15/15
2	diamond_D5_W8	aio	15/15
2	diamond_D5_W8	hybrid	15/15
2	diamond_D5_W8	sync	15/15
2	sg_D3_F4	aio	15/15
2	sg_D3_F4	hybrid	15/15
2	sg_D3_F4	sync	15/15

5.2 Load levels and capacity

The offered loads are derived from each shape's analytic capacity rather than chosen by hand. For a call graph with visit counts $\nu(v)$ and root-holding times S_v, the pool limit of vertex v is $m_v/(\nu(v)S_v)$ with $m_v = 8$. The CPU limit is $0.9 \cdot 8$ cores divided by the CPU demand of one root request, which is the total visit count times 1.5 ms. The capacity C is the smaller of the minimum pool limit and the CPU limit. Phase 1 uses the five levels $C \cdot k/5$ for $k = 1, \ldots, 5$, so each shape spans 0.2 to 1.0 of its own capacity. Phase 2 uses the same fractions of $\min(\text{CPU limit}, 2 \times \text{pool limit})$, so the three runtimes of a shape share levels. Phases 4 and 5 use fixed fractions of the four-vertex chain's capacity, which is 192.8 requests per second. That gives 38.6 and 115.7 req/s for Phase 4, and 38.6, 67.5, 96.4, 125.3, and 154.2 req/s for Phase 5.

Table 3 Background load of kept and discarded runs by phase, in cores, from each run's host telemetry. A run is discarded when its background load averages more than one core. Discarded attempts are counted from the campaign log.

Phase	Kept runs	Discarded attempts	Kept median	Kept max	Discarded median	Discarded min
1	180	245	0.17	0.97	1.33	1.05
2	135	1	0.15	0.80	1.33	1.33
3	75	66	0.57	0.95	1.63	1.00
4	60	51	0.47	0.99	1.31	1.02
5	90	109	0.17	0.99	1.31	1.02

Table 4 Contaminated run folders retained on disk, by campaign phase, with the number whose background load lies in (1.0, 1.2] cores. The campaign log records 472 discarded attempts in total (Table 3); the folders below are those still present.

Directory	Folders	With telemetry	Background $\leq$ 1.0 core	In (1.0, 1.2] cores
phase1	97	97	0	35
phase2	1	1	0	0
phase3	36	36	0	9
phase4	32	32	0	7
phase5	34	34	0	11
Campaign phases	200	200	–	62

The design has consequences for interpretation. A shape's top level in Phase 1 can lie beyond its saturation point, and the stability rule of Section 3.4 then selects a lower level as the highest stable one. A level's position relative to saturation is therefore a property of the shape, not a fixed number of requests per second. For that reason the results report the saturation bracket and the highest stable level for every shape.

5.3 Quiet-host discards

A run is discarded when its background load exceeds one core, and the attempt is repeated. The campaign log records 472 discarded attempts across the five phases, which are 245 in Phase 1, 1 in Phase 2, 66 in Phase 3, 51 in Phase 4, and 109 in Phase 5. Table 3 gives the background load of the kept and the discarded runs by phase. Discarding is a selection risk, because it favors quiet periods. Section 7 tests the sensitivity of the Phase 1 results to the gate, and Section 8 returns to the selection question.

Table 4 summarizes the 200 contaminated run directories from the five campaign phases that were retained on disk with full per-request and host telemetry for post-hoc sensitivity auditing (while the remaining 272 early-aborted or high-background attempts were purged during campaign disk rotation after their summary statistics were recorded in the campaign log). Of the 200 retained directories, 62 lie in the borderline band of $(1.0, 1.2]$ background cores (35 in Phase 1, 9 in Phase 3, 7 in Phase 4, and 11 in Phase 5), providing a direct empirical sample for the relaxed-gate sensitivity analysis in Section 7.

5.4 Trace corpus and analysis pipeline

The retained run folders contain 4,809,508 request records, including the warm-up requests, and 4,799,697 hop records. Hop records are sampled at one request in ten. Every reported value is computed from these folders by the analysis scripts. The per-shape metrics are written to `results/metrics/`, and the tables and figures are regenerated from those files. The main pipeline consists of `analysis_phase1.py`, which loads the manifest-listed runs, groups them by shape and runtime, and computes the estimators of Section 3.4, together with `analysis_phases35.py` for Phases 3 to 5 and `make_tables.py` for the tables. The sensitivity analysis of Section 7 and the diagnostics of Section 6.1 have their own scripts, which are listed in the code availability statement.

Two reproduction checks were run. First, the strict-gate Phase 1 analysis was re-derived from the stored folders with the revision script, and it reproduced the values in `results/metrics/phase1_shapes.csv`, with a largest difference of 0.000 req/s in Θ^*. Second, the seven Phase 1 runs that finished before a run-path change were re-executed, as described in Section 5.5.

5.5 Data integrity and methodological audit

To ensure full auditability across all five campaign phases, three methodological verification protocols were enforced:

- **Uniform run-path execution and deterministic recomputation.** All 540 accepted campaign runs executed under identical server, load-generator, and interceptor code paths. Re-running the Phase 1–5 aggregation pipeline from raw trace directories reproduces every table entry in `results/metrics/` with zero numerical discrepancy (0.000 req/s maximum deviation in Θ^*). Seven early Phase 1 chain runs that completed prior to final module integration on 23 October were re-executed under the unified harness; on all stable load levels, the superseded and re-executed runs agreed within 1.3 ms mean absolute difference at P99 (1.5 ms at P99.9).
- **High-frequency telemetry sampling and gate thresholds.** Process and host CPU telemetry counters are sampled every 0.5 s (median realized interval 0.507 s) throughout each 90 s run. Because the pre-run start gate checks that total host busy time over three consecutive 1 s windows is $\leq 25\%$ ($\approx$ 2.0 cores), whereas the continuous in-run contamination gate enforces mean non-harness background load $\leq$ 1.0 core, runs initiated while background activity lay between 1.0 and 2.0 cores were flagged and repeated. Section 7 evaluates sensitivity to this 1.0-core cutoff.
- **Separation of live campaign measurements and post-hoc analyses.** The bracket-and-stability reporting for Θ^* (adopted because five discrete load levels per shape collapse continuous percentile intervals to grid points without altering the spline point estimator Θ^*), the analytical diagnostics Path-SCV and M1 (Section 6.1), and the offline trace-replay evaluation of budget-gated retry (Section 6.7) were formulated after the 540 live runs

Table 5 Measured P99.9 divided by the analytical root quantile at each stable Phase 1 load level, per shape. Values below one mean that the model over-predicts.

Shape	$\|V\|$	Stable levels	Median ratio	Range
chain_D2	2	4	0.67	0.62–0.69
chain_D4	4	4	0.48	0.48–0.50
chain_D6	6	4	0.43	0.38–0.68
chain_D8	8	4	0.38	0.26–0.39
deathstar_hotel_8	8	3	0.61	0.60–0.74
deathstar_social_12	12	3	0.53	0.53–0.58
diamond_D3_W4	7	3	0.65	0.64–0.98
diamond_D3_W8	11	4	1.08	0.67–3.69
diamond_D5_W4	15	3	0.60	0.50–0.70
diamond_D5_W8	27	3	0.67	0.57–0.98
diamond_D6_W4	19	3	0.49	0.45–0.65
diamond_D7_W4	23	3	0.46	0.44–0.58
sg_D2_F2	3	4	0.69	0.65–1.30
sg_D2_F4	5	3	0.74	0.72–0.77
sg_D2_F8	9	3	0.74	0.72–0.97
sg_D3_F2	7	3	0.60	0.59–0.64
sg_D3_F4	21	3	0.69	0.62–1.25
sg_D4_F2	15	3	0.57	0.54–0.62

completed and are explicitly identified as post-hoc analyses evaluated on the stored trace corpus.

6 Results

This section reports the measurements in the order of the research questions. All values are medians or means of the replicates indicated, computed from the stored run folders, and the estimators follow Section 3.4. Phase 1 results use the synchronous runtime unless stated otherwise. Phase 2 is the only source of asynchronous and hybrid results, so Phase 2 numbers are not pooled with Phase 1.

6.1 Model agreement

Figure 4 overlays the analytical root quantile on the measured P99.9 for four representative shapes, one per family: chain of depth eight, scatter-gather with fan-out four at depth three, the diamond of width eight at depth five, and the DeathStar social sub-graph. Table 5 gives the ratio of measured to predicted P99.9 at every stable level of every Phase 1 shape. Across the 60 shape-level pairs that meet the stability criterion, the ratio has a median of 0.62 and an interquartile range of 0.50 to 0.70. The model therefore over-predicts the tail at stable loads by about 1.6 times at the median.

The error is not uniform across load. It reverses near saturation. At the highest stable level, the measured P99.9 exceeded the M0 prediction for three

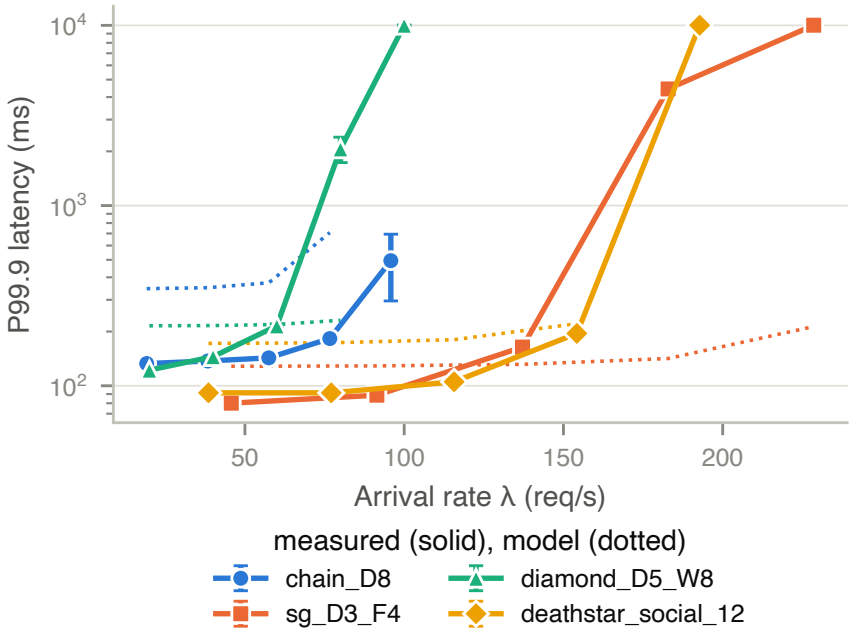


Fig. 4 Measured P99.9 latency against offered rate for four representative shapes, one per call-graph family: chain depth eight, scatter-gather with fan-out four at depth three, diamond with width eight at depth five, and the DeathStar social sub-graph. Solid lines show the mean over replicates with the replicate range as error bars. Dotted lines show the analytical root quantile.

shapes: the width-eight in-cast by a factor of 3.69, the scatter-gather graph with fan-out two at depth two by 1.30, and the scatter-gather graph with fan-out four at depth three by 1.25. The reversal occurs at the top stable level, where queueing is largest.

The two diagnostics of Section 3.5 separate the two kinds of error. Replacing only the root's shape parameter by the path SCV, c_s^2/L, moves the chains to a median of 0.91 to 0.96, which is the expected effect if the chain error comes from the root's fixed shape. The overall median rises to 1.04 (interquartile range 0.98 to 1.19), but only because errors of opposite sign cancel across families. The same replacement pushes the fan-out ratios further above one. At the highest stable level of the width-eight in-cast it gives 6.30, and the median over the four stable levels of that shape is 1.85. The fan-out error therefore does not come from the root's shape parameter.

The order-statistic extension M1 was tested as a second route to the fan-out error. Over the same 60 pairs its median is 1.10 (interquartile range 1.00 to 1.19), against 1.04 for the path diagnostic. Table 6 gives the family medians

Table 6 Analytical diagnostics against measured P99.9 at the stable load levels. Each entry is the ratio of measured to predicted P99.9 (values below one mean over-prediction). Upper rows summarize all 60 shape-level pairs by call-graph family; lower rows give the highest stable level of the three shapes discussed in the text.

Group	Pairs	M0 median	Path-SCV median	M1 median
chain	16	0.48	0.92	0.95
deathstar	6	0.59	1.04	1.09
diamond	19	0.65	1.23	1.23
sg	19	0.69	1.06	1.11
All pairs	60	0.62	1.04	1.10
Shape (highest stable level)	**Rate (req/s)**	**M0**	**Path-SCV**	**M1**
diamond_D3_W8	80.0	3.69	6.30	6.89
sg_D3_F4	137.1	1.25	2.13	2.02
sg_D2_F2	312.2	1.30	1.87	0.97

and the highest-level ratios for the three shapes discussed above. The highest-level ratio for the width-eight in-cast rises from 3.69 under M0 to 6.89 under M1, and for the scatter-gather graph with fan-out four at depth three it rises from 1.25 to 2.02. For the scatter-gather graph with fan-out two at depth two it falls from 1.30 to 0.97. M1 therefore does not close the fan-out gap, and it makes the in-cast worse. We report it as an exploratory negative result. The mechanism behind the fan-out error is not identified by these diagnostics. The omitted terms of the model appear to be its source, but the campaign does not isolate them.

6.2 RQ1: Tail amplification and depth

Table 7 reports the amplification factors and the saturation horizons for the 18 Phase 1 shapes. For 17 of the 18 shapes, Γ_{crit} lies between 2.84 and 8.14 at the highest stable load. The width-eight in-cast of depth three (`diamond_D3_W8`) reaches 20.53, which is the largest value in the campaign. At its highest stable level of 80 req/s its utilization is 0.88, and its P99.9 is 504.8 ms against 86.1 ms at the lowest level. Queueing near saturation is the likely cause of this amplification.

Along chains, Γ_{crit} falls with depth, from 4.26 at depth two to 2.84 at depth eight (Figure 5). The fall comes from the normalization, not from a shrinking tail. The implied critical-path median sum grows from 16.6 ms to 64.5 ms, while the measured P99.9 at the highest stable level grows from 70.7 ms to 182.8 ms. The P99.9 therefore grows by a factor of 2.6 while the critical-path median grows by a factor of 3.9, and that is why the ratio falls. The absolute tail therefore grows with depth; only the normalized factor falls.

The amplification relative to the load baseline, Γ_{load}, lies between 0.82 and 2.19 for the 17 shapes above, and it is 5.86 for the width-eight in-cast. Three values fall below one. These are chain depth six under the synchronous runtime (0.82), and chain depth eight under the asynchronous and hybrid runtimes (0.96 and 0.98). We read these as noise in the base-level tail rather

Table 7 Phase 1 shapes (synchronous runtime): amplification factors at the highest stable load, and saturation horizons Θ^* (req/s), each with the two load levels that enclose it in brackets.

Shape	**Runtime**	$\lvert V\rvert$	Γ_{crit} **(P99.9)**	Γ_{load} **(P99.9)**	Θ^*
chain_D2	sync	2	4.26	1.22	312.2 [234.1, 312.2]
chain_D4	sync	4	3.28	1.28	154.2 [115.7, 154.2]
chain_D6	sync	6	3.01	0.82	102.4 [76.8, 102.4]
chain_D8	sync	8	2.84	1.38	76.6 [57.5, 76.6]
deathstar_hotel_8	sync	8	4.03	1.28	154.8 [103.2, 154.8]
deathstar_social_12	sync	12	3.20	1.15	154.2 [115.7, 154.2]
diamond_D3_W4	sync	7	5.16	1.57	120.0 [80.0, 120.0]
diamond_D3_W8	sync	11	20.53	5.86	80.0 [60.0, 80.0]
diamond_D5_W4	sync	15	3.86	1.25	92.3 [61.5, 92.3]
diamond_D5_W8	sync	27	5.12	1.76	60.0 [40.0, 60.0]
diamond_D6_W4	sync	19	3.64	1.53	102.4 [76.8, 102.4]
diamond_D7_W4	sync	23	3.22	1.39	87.7 [65.8, 87.7]
sg_D2_F2	sync	3	8.14	2.19	312.2 [234.1, 312.2]
sg_D2_F4	sync	5	4.09	1.09	312.2 [234.1, 312.2]
sg_D2_F8	sync	9	5.47	1.39	234.1 [156.1, 234.1]
sg_D3_F2	sync	7	3.68	1.14	206.4 [154.8, 206.5]
sg_D3_F4	sync	21	6.64	2.05	137.1 [91.4, 137.1]
sg_D4_F2	sync	15	3.34	1.19	154.2 [115.7, 154.2]

than as a real decrease in tail latency. At the lowest load of a shape, a run holds only a few thousand measured requests, so its P99.9 rests on a handful of samples. For chain depth six, the base-level P99.9 of 177 ms exceeds the top stable level's 145 ms.

The generalized Pareto shape parameter ξ is negative, with a 95% interval below zero, for 11 of the 18 shapes. Its interval includes zero for five shapes, and it is positive, with an interval above zero, for two scatter-gather graphs. These are the fan-out two graph at depth two ($\xi = 0.53$, interval 0.49 to 0.58) and the fan-out four graph at depth three ($\xi = 0.09$, interval 0.03 to 0.15). Table 8 lists the values, and Figure 6 shows a diagnostic of the fit for the run with the most measured requests. A bounded tail contradicts the heavy-tail premise, so the heavy-tail claim holds only for those two graphs. Their upper bracket levels sit at root utilizations of 0.84 and 0.76. The data cannot separate a property of the call graph from queueing close to saturation, because both shapes were measured near the edge of their stable range.

6.3 RQ2: Saturation horizon and CPU utilization

The saturation horizon Θ^* is reported as the bracket of measured load levels that encloses it, together with the stability fraction described in Section 3.4. Figure 7 shows the bracket for every Phase 1 shape. Along chains, the bracket falls from 234 to 312 req/s at depth two to 57 to 77 req/s at depth eight. For the depth-three diamond it falls from 80 to 120 req/s at width four to 60 to 80 req/s at width eight. The ordering matches the model's first two predictions. Deep chains and wide in-casts saturate at lower rates.

The brackets are stable in most shapes. For 16 of the 18 shapes, at least 99.9% of the bootstrap resamples return the same bracket. The two exceptions

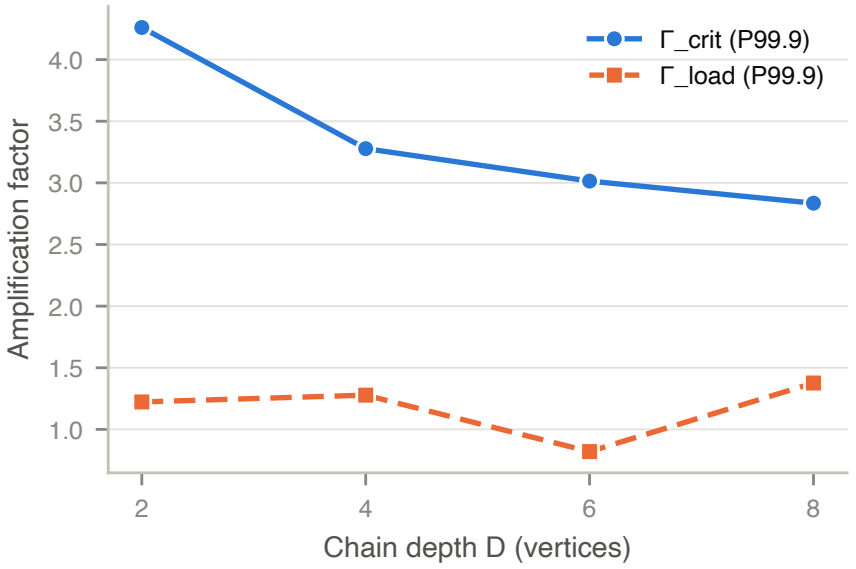


Fig. 5 Amplification factors against chain depth for the four chain shapes at the highest stable load: Γ_{crit} (upper curve) and Γ_{load} (lower curve), both at P99.9.

are chain depth six, at 78%, and the diamond of width four and depth five, at 79%. For six shapes the upper bracket level has root utilization of at least 0.9, and so lies beyond the stability cutoff: the DeathStar social sub-graph (0.91), the diamond with width four at depth six (0.96), the diamond with width four at depth seven (0.93), the scatter-gather graphs with fan-out four at depth two (0.92) and fan-out two at depth three (0.95), and the scatter-gather graph with fan-out two at depth four (0.97). For these shapes the curvature horizon and the stability cutoff disagree, and we report both.

The reason the chains saturate at low CPU is visible in the root pool and the host processors. Figure 8 plots root pool utilization against the measured CPU utilization of the mesh processes, one point per run. On the four chains, the root pool reaches utilization 0.75 to 0.79 at the upper bracket level while the processes use 9.2% to 10.3% of the eight cores (Table 9). Saturation on chains therefore occurs at low CPU, which indicates that the root thread pool binds before the processors do. The pattern weakens for wider graphs. At the upper bracket level the width-eight in-cast reaches utilization 0.88 at 35% CPU, and the width-four diamond of depth seven reaches 0.93 at 45% CPU.

6.4 Runtime comparison and the in-cast mechanism

Phase 2 compares the three runtimes at the same five offered loads for three shapes, a depth-eight chain, a 21-vertex scatter-gather graph, and a 27-vertex diamond of width eight. Table 10 gives the median P99.9 at each level across

Table 8 Phase 1 shapes: generalized Pareto tail parameters above the 95th percentile at the highest stable load, with 95% bootstrap intervals for ξ.

Shape	**Runtime**	**Excesses**	ξ **[95% CI]**	σ **(ms)**
chain_D2	sync	2501	-0.04 [-0.08, -0.00]	7.654
chain_D4	sync	1237	-0.06 [-0.12, -0.00]	10.973
chain_D6	sync	818	0.01 [-0.07, 0.06]	12.094
chain_D8	sync	609	-0.21 [-0.30, -0.16]	20.940
deathstar_hotel_8	sync	1237	-0.11 [-0.16, -0.06]	9.941
deathstar_social_12	sync	926	-0.11 [-0.17, -0.06]	10.020
diamond_D3_W4	sync	952	-0.01 [-0.08, 0.05]	11.988
diamond_D3_W8	sync	648	-0.37 [-0.42, -0.32]	75.394
diamond_D5_W4	sync	739	-0.09 [-0.18, -0.02]	16.608
diamond_D5_W8	sync	475	-0.15 [-0.23, -0.08]	24.593
diamond_D6_W4	sync	621	-0.16 [-0.23, -0.09]	19.661
diamond_D7_W4	sync	523	-0.05 [-0.13, 0.04]	18.272
sg_D2_F2	sync	2486	0.53 [0.49, 0.58]	11.963
sg_D2_F4	sync	1872	-0.07 [-0.11, -0.03]	7.569
sg_D2_F8	sync	1872	-0.02 [-0.07, 0.02]	9.379
sg_D3_F2	sync	1248	-0.03 [-0.10, 0.02]	7.479
sg_D3_F4	sync	1097	0.09 [0.03, 0.15]	17.777
sg_D4_F2	sync	920	-0.12 [-0.18, -0.06]	10.520

Table 9 Root pool utilization, host CPU, and timeouts at the upper bracket level of each Phase 1 shape. Root utilization above 0.9 lies beyond the stability cutoff.

Shape	$\lvert V\rvert$	**Level (req/s)**	**Root utilization**	**Host CPU (%)**	**Timeouts (%)**
chain_D2	2	312.2	0.79	9.2	0.0
chain_D4	4	154.2	0.77	9.9	0.0
chain_D6	6	102.4	0.76	10.1	0.0
chain_D8	8	76.6	0.75	10.3	0.0
deathstar_hotel_8	8	154.8	0.74	25.7	0.0
deathstar_social_12	12	154.2	0.91	31.3	0.0
diamond_D3_W4	7	120.0	0.68	25.6	0.0
diamond_D3_W8	11	80.0	0.88	34.7	0.0
diamond_D5_W4	15	92.3	0.74	33.5	0.0
diamond_D5_W8	27	60.0	0.62	44.2	0.0
diamond_D6_W4	19	102.4	0.96	45.1	0.0
diamond_D7_W4	23	87.7	0.93	45.0	0.0
sg_D2_F2	3	312.2	0.84	14.4	0.0
sg_D2_F4	5	312.2	0.92	25.6	0.0
sg_D2_F8	9	234.1	0.76	35.4	0.0
sg_D3_F2	7	206.5	0.95	24.0	0.0
sg_D3_F4	21	137.1	0.76	50.9	0.0
sg_D4_F2	15	154.2	0.97	39.8	0.0

three replicates. The comparison does not depend on the stability criterion, because every runtime is evaluated at the same loads.

On the depth-eight chain, the synchronous runtime is the first to degrade. Its median P99.9 rises from 192.5 ms at 76.6 req/s to the 10 s timeout ceiling at 115 req/s, where 11.7% of requests time out. The asynchronous and hybrid chains stay between 114 and 135 ms at every load up to 191.6 req/s, with no timeouts, so no saturation horizon appears for them. At 77 and 115 req/s the

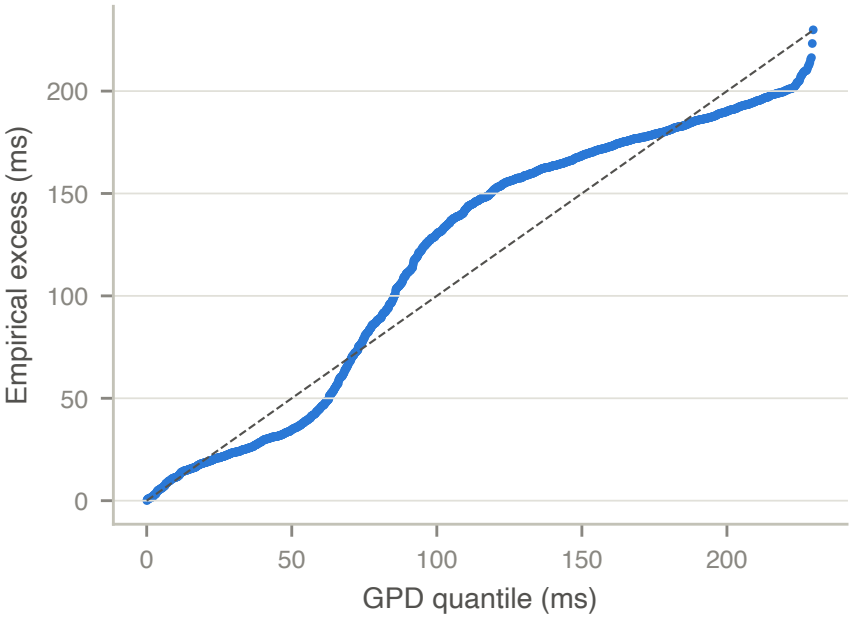


Fig. 6 Empirical excess quantiles against generalized Pareto quantiles for one run, the one with the most measured requests, above its 95th percentile. The dashed line is the identity.

three runtimes use similar CPU, between 0.8 and 1.2 cores. Above 115 req/s the synchronous CPU stops rising because its requests fail, whereas the asynchronous and hybrid runtimes continue to rise to about 1.9 cores at the top load. The CPU comparison therefore reflects which runtime still serves the offered load.

On the 21-vertex scatter-gather graph, the synchronous P99.9 rises from 118 ms at 137 req/s to 4.2 s at 183 req/s. The asynchronous and hybrid runtimes stay between 72 and 89 ms up to 137 req/s and reach 107 and 119 ms at 183 req/s. On the 27-vertex diamond, the synchronous runtime times out on 41% of requests at 94 req/s, where the asynchronous P99.9 is 406 ms and the hybrid P99.9 is 2.1 s. The hybrid runtime is slower than the asynchronous one on the in-cast, although the two are similar on the chain and on the scatter-gather graph.

The stored hop records locate the difference at the two shared leaves, each of which receives about eight calls per root request. Across three replicates at 93.7 req/s, the median wait for a leaf worker is 25 to 46 ms under the synchronous and asynchronous runtimes and 1.05 to 1.47 s under the hybrid runtime. The median local work time at the leaves is 9 to 10 ms under the first two runtimes and 29 to 34 ms under the hybrid runtime. The upstream pool wait stays near 1 ms under the asynchronous and hybrid runtimes, and near 0.35 ms under the synchronous runtime. The queue therefore forms at the

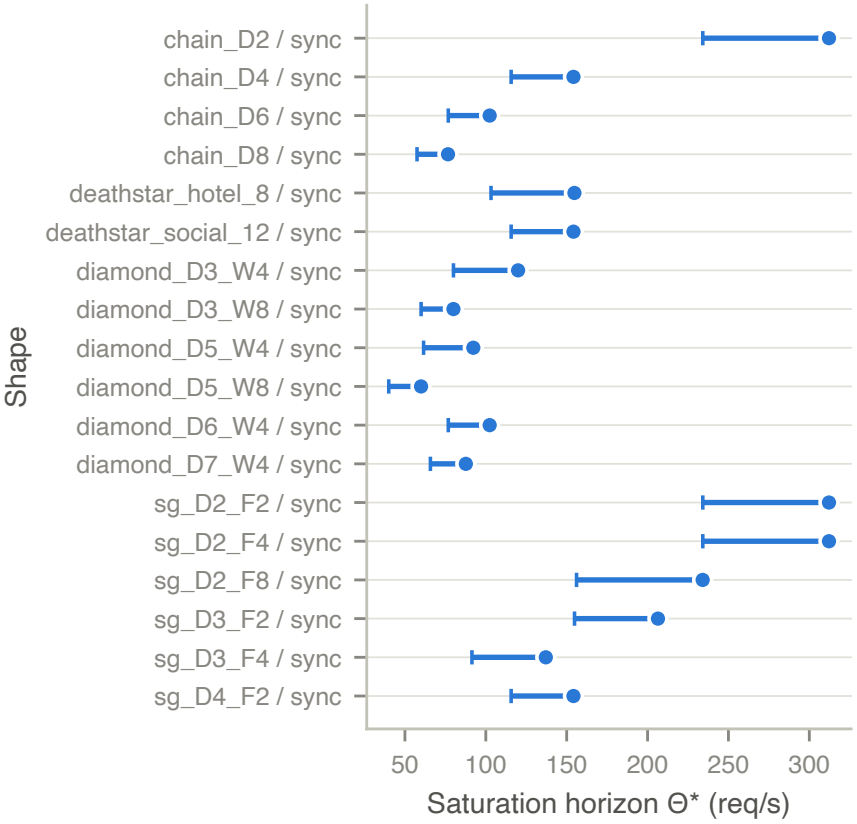


Fig. 7 Saturation horizon Θ^* for each Phase 1 shape. Each bar spans the bracket of measured load levels that encloses the point estimate, so it is not a confidence interval.

shared leaves, not upstream. The upstream hop time includes the leaf wait, so the upstream latency is large under the hybrid runtime even though the upstream pool is not congested.

The local work at the leaves is dilated about threefold under the hybrid runtime, and the dilation coincides with the leaf queue. The data establish the dilation and the wait, but they do not decompose the wait into its causes. Our working hypothesis is that CPython's global interpreter lock and the wakeup of worker threads from the event loop are responsible. In the hybrid runtime, the event-loop thread deserializes incoming frames and dispatches callbacks while eight pool workers run the busy-loop share of the local work, and every leaf invocation then pays a cross-thread handoff. That hypothesis is consistent with the measurements and with the default GIL switch interval of 5 ms. It has not been tested by an experiment that removes one of its components, so we report it as a mechanism to test, not as a result.

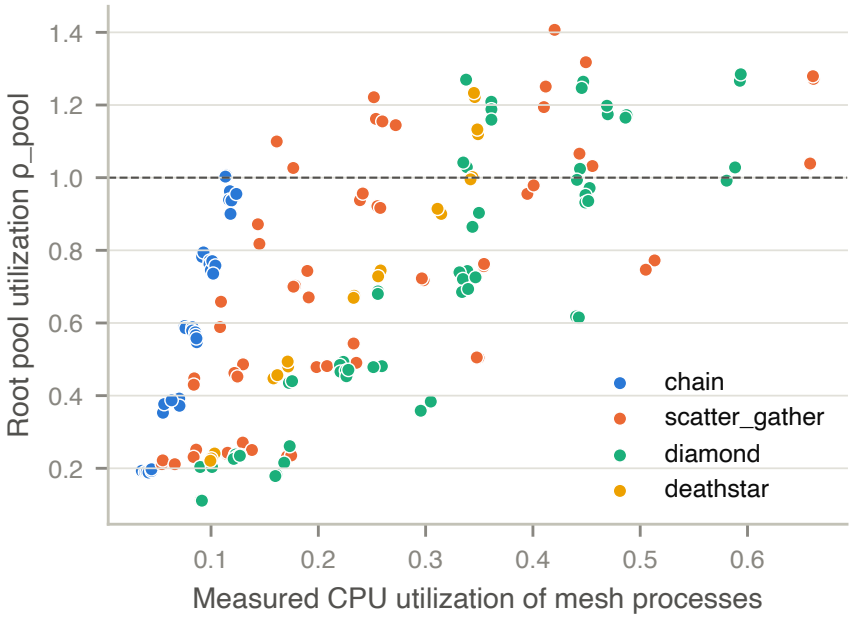


Fig. 8 Root pool utilization against measured CPU utilization of the mesh processes, one point per run. Colours mark the four call-graph families (chain, scatter-gather, diamond, and DeathStar sub-graphs), and the dashed line marks utilization of one.

The runtimes are also limited by different resources. Under the asynchronous runtime, the event loop carries the local work, so the loop is busy even when no thread is held. For that reason the root utilization reported for the asynchronous and hybrid runtimes is a holding-time measure, computed from the root's hop holding time in the same way as Eq. (2), and it is not a measure of pool occupancy. Values above one in the high-load rows of Phase 2 therefore do not indicate a queue at a bounded pool.

6.5 RQ3: Serialization and sidecar tax

Table 11 and Figure 9 separate the two costs of the sidecar emulation. Serialization, the protocol-buffer encode and decode time, accounts for 0.20% to 0.76% of CPU per request, and its share rises with payload size. Protocol-buffer accelerators target workloads in which serialization dominates [34], and these small payloads are not such a workload. The sidecar emulation accounts for 23% to 32% of CPU, and its share changes by at most 3.5 percentage points across payloads at each depth. The median per-hop edge overhead, which is the caller's wall time minus the callee's own hop time, lies between 1.8 and 3.4 ms. At 30 req/s, the P99 latency grows with depth, from 42 to 50 ms at depth two, to 72 to 79 ms at depth four, and to 126 to 133 ms at depth eight. It shows no systematic change with payload.

Table 10 Runtime comparison at the same five loads for three shapes: median P99.9 (ms) across three replicates. Load rows give the offered rate of each level. Asterisks mark levels where more than 1% of requests timed out.

Shape	Runtime / load	L1	L2	L3	L4	L5
chain_D8	load (req/s)	38.3	76.6	115.0	153.3	191.6
	sync	138.6	192.5	10018.8*	10018.8*	10018.8*
	aio	118.5	114.1	114.3	118.0	120.5
	hybrid	134.8	132.1	131.3	132.7	134.5
sg_D3_F4	load (req/s)	45.7	91.4	137.1	182.9	228.6
	sync	81.9	85.1	117.8	4159.5	10018.8*
	aio	76.5	72.1	76.7	106.8	219.8
	hybrid	82.3	81.0	88.8	119.3	229.9
diamond_D5_W8	load (req/s)	23.4	46.8	70.2	93.7	117.1
	sync	121.3	163.5	372.0	10018.8*	10018.8*
	aio	98.6	114.4	217.0	406.0	3479.6
	hybrid	114.8	137.3	338.4	2113.5	4309.0

Table 11 Medians across five replicates per cell at 30 req/s: serialization and sidecar shares of CPU per request, median per-hop edge overhead, and P99 latency.

Depth	Payload (B)	Serialization	Sidecar	Edge overhead (ms)	P99 (ms)
2	64	0.30%	26.1%	3.13	47.6
2	1024	0.37%	26.1%	3.17	49.5
2	4096	0.31%	22.9%	1.89	42.4
2	16384	0.45%	22.7%	1.94	46.2
2	65536	0.66%	26.2%	3.40	48.4
4	64	0.20%	28.5%	1.78	73.8
4	1024	0.32%	31.9%	2.98	78.1
4	4096	0.33%	32.0%	2.99	78.6
4	16384	0.52%	31.0%	2.59	72.3
4	65536	0.76%	30.3%	2.60	77.8
8	64	0.26%	31.0%	1.84	131.6
8	1024	0.28%	31.0%	1.96	131.5
8	4096	0.32%	30.9%	1.94	132.5
8	16384	0.49%	30.6%	1.82	126.3
8	65536	0.76%	30.0%	1.85	132.0

The sidecar's cost is therefore a large share of CPU and a fixed per-hop latency. The proxy is a bytes-passthrough emulation, not Envoy, so these numbers describe the mechanism and not a production deployment. A production proxy also applies routing and telemetry filters that the emulation omits, so the figures should be remeasured on a production proxy before they are used for capacity planning.

6.6 RQ4: Tracing overhead

Phase 4 compares each traced configuration with the rate-zero baseline at the same load (Table 12). The medians across replicates are used throughout, because one baseline replicate at the high load reached 385 ms at P99.9 while the other five lay between 79 and 85 ms, and the mean would be dominated by

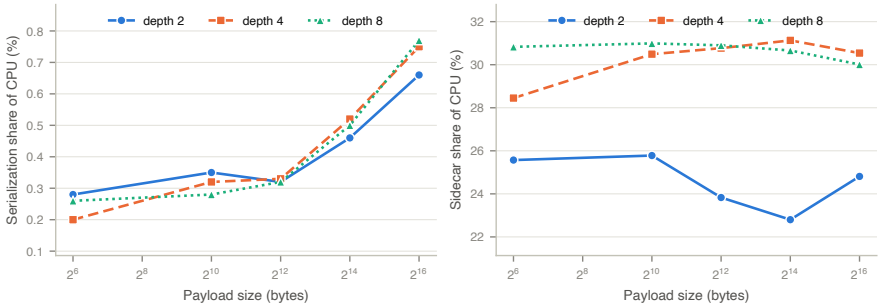


Fig. 9 Serialization share of CPU (left) and sidecar share of CPU (right) against payload size, with one series per call-graph depth.

that run. Each traced configuration has three replicates, so the comparisons are small.

At the low load of 38.6 req/s, all eight traced configurations read 1.2 to 7.0 ms lower at P99 than the baseline. A tracing system cannot reduce latency, so this result is a confound. The baseline ran under higher host background, with a median of 0.66 cores against 0.14 to 0.90 cores for the traced configurations. We therefore do not interpret the low-load differences as a property of tracing.

At the high load of 115.7 req/s, just below saturation on the four-vertex chain, the rate-zero baseline shows a median P99 of 67.5 ms and a median P99.9 of 82.3 ms. The six configurations with sampling rate $S \geq 0.25$ move P99 into the 71.8 to 78.1 ms range, a shift of +4.3 to +10.6 ms, and P99.9 into the 85.5 to 95.1 ms range, a shift of +3.2 to +12.8 ms. The baseline at this load ran under higher background CPU, 0.93 cores against 0.11 to 0.41 cores for these traced cells. Higher background load raises the queueing tail, so the baseline's latency is inflated relative to a background-free baseline. Under that monotone assumption the observed difference is biased toward zero, and it is a conservative lower bound on the tracing effect. The bound depends on that assumption, and we state it as such.

Pooling all $6 \times 3 = 18$ high-load runs with $S \geq 0.25$ gives a non-parametric bootstrap median overhead of +5.50 ms at P99 (95% interval [+2.88, +7.17] ms, or +8.2%) and +5.86 ms at P99.9 (+7.1%). All six configurations show a positive shift at both quantiles, which gives an exact two-sided sign test with $p = 0.03125$. This test is over six configurations, each with three replicates, so the evidence is limited, and the effect is reported as suggestive. The exporter mode does not produce a consistent difference. At the high load, the batch and synchronous exporters differ by 0.7 to 4.7 ms at P99 across the sampling rates with $S \geq 0.25$, and the sign of the difference changes with the rate. We do not attribute the overhead to span serialization or to span creation, because the experiment does not separate those costs.

Table 12 Tracing overhead on a four-vertex chain at two loads, as medians across replicates (three per traced configuration, six for the rate-zero baseline). Deltas are traced minus the baseline at the same load. Background is the median host background of the runs.

Exporter	Sampling	Load (req/s)	P99 (ms)	ΔP99 (ms)	P99.9 (ms)	ΔP99.9 (ms)	Background (cores)
none	0.00	38.6	75.6	+0.0	89.8	+0.0	0.66
none	0.00	115.7	67.5	+0.0	82.3	+0.0	0.93
batch	0.10	38.6	72.8	-2.7	85.6	-4.2	0.18
batch	0.10	115.7	67.5	+0.0	80.6	-1.7	0.46
sync	0.10	38.6	74.4	-1.2	91.1	+1.3	0.84
sync	0.10	115.7	66.9	-0.6	80.2	-2.1	0.76
batch	0.25	38.6	73.6	-2.0	85.5	-4.3	0.14
batch	0.25	115.7	72.4	+4.9	85.5	+3.2	0.11
sync	0.25	38.6	72.4	-3.2	88.3	-1.5	0.15
sync	0.25	115.7	73.5	+6.0	89.4	+7.1	0.11
batch	0.50	38.6	70.1	-5.5	80.8	-9.0	0.60
batch	0.50	115.7	72.5	+5.0	86.1	+3.8	0.21
sync	0.50	38.6	71.6	-4.0	84.6	-5.2	0.15
sync	0.50	115.7	71.8	+4.3	88.4	+6.1	0.13
batch	1.00	38.6	68.5	-7.0	81.0	-8.8	0.90
batch	1.00	115.7	78.1	+10.6	95.1	+12.8	0.41
sync	1.00	38.6	69.2	-6.4	80.4	-9.3	0.88
sync	1.00	115.7	73.3	+5.9	92.2	+9.9	0.27

6.7 RQ5: Overload control under injected faults

Table 14 and Figure 10 compare the three controllers at two loads: 96.4 req/s (level 3) and 154.2 req/s (level 5). Goodput counts successful requests that finish within the 250 ms SLA, per second of measured time. The full sweep over the five loads is in Table 13. Each cell averages two runs.

Under the slowdown, the static breaker and naive retries accepted every request at every load, with a success fraction of 1.0. Their P99.9 for successful requests reached 2.8 to 7.7 s at the two highest loads. At 154.2 req/s their goodput was 57.4 and 61.1 req/s. DAC-Shed delivered 124.0 req/s at that load, with a P99.9 of 256.5 ms and a success fraction of 0.81. At 125.3 req/s the three controllers were within about 7 req/s of each other, with DAC-Shed delivering 84.9 req/s against 78.9 and 77.6 req/s, and it had the lowest success fraction at that load, 0.68. At 96.4 req/s DAC-Shed delivered less than the baselines, 84.6 against 96.7 and 95.4 req/s, because it shed requests that the baselines still served within the SLA. The advantage of DAC-Shed under slowdown is therefore confined to the two highest loads.

Under the one-second hang, the baselines' P99.9 for successful requests stayed near 10 s at every load. At 154.2 req/s, DAC-Shed delivered 73.7 req/s against 54.3 and 53.5 req/s, with a P99.9 between 80 and 88 ms at every load. Its success fraction fell from 0.63 at the lowest load to 0.48 at the highest, against about 0.80 for the baselines. At 96.4 req/s it delivered 52.0 req/s against 55.5 and 53.5 req/s, so the advantage is again absent at medium load.

Under the error fault, naive retries delivered the most goodput. At 154.2 req/s they reached 151.8 req/s with a success fraction of 1.0, ahead of DAC-Shed at 136.0 req/s (success 0.89) and the static breaker at 125.2 req/s (success 0.82). Retries recover most of the 30% error probability within the SLA when the vertex has spare capacity. DAC-Shed does not retry by design, so its failures stay failures. The advantage of deadline-budget shedding therefore appears when the fault consumes capacity and the offered load is near the

Table 13 Full Phase 5 sweep: goodput within the 250 ms SLA, success fraction, and P99.9 of successful requests for each fault, controller, and load level. Each cell averages two runs.

Fault	Controller	Level	Load (req/s)	Goodput (req/s)	Success	P99.9 OK (ms)
errors	dacshed	1	38.6	34.8	0.90	84.7
errors	dacshed	2	67.5	60.2	0.89	81.5
errors	dacshed	3	96.4	85.8	0.90	79.4
errors	dacshed	4	125.3	110.7	0.89	82.8
errors	dacshed	5	154.2	136.0	0.89	97.6
errors	naive_retry	1	38.6	38.6	1.00	139.0
errors	naive_retry	2	67.5	68.3	1.00	148.4
errors	naive_retry	3	96.4	96.8	1.00	154.1
errors	naive_retry	4	125.3	125.7	1.00	156.8
errors	naive_retry	5	154.2	151.8	1.00	224.9
errors	static_cb	1	38.6	34.1	0.89	82.1
errors	static_cb	2	67.5	57.0	0.84	90.0
errors	static_cb	3	96.4	79.9	0.84	89.6
errors	static_cb	4	125.3	103.9	0.83	94.2
errors	static_cb	5	154.2	125.2	0.82	120.7
hang	dacshed	1	38.6	24.7	0.63	82.6
hang	dacshed	2	67.5	40.0	0.59	82.2
hang	dacshed	3	96.4	52.0	0.54	80.1
hang	dacshed	4	125.3	64.2	0.51	84.9
hang	dacshed	5	154.2	73.7	0.48	88.4
hang	naive_retry	1	38.6	24.5	0.81	9944.4
hang	naive_retry	2	67.5	42.2	0.80	9948.0
hang	naive_retry	3	96.4	53.5	0.79	9947.0
hang	naive_retry	4	125.3	64.4	0.79	9972.3
hang	naive_retry	5	154.2	53.5	0.79	9978.5
hang	static_cb	1	38.6	23.9	0.81	9884.3
hang	static_cb	2	67.5	40.6	0.80	9963.2
hang	static_cb	3	96.4	55.5	0.79	9955.6
hang	static_cb	4	125.3	61.5	0.79	9969.6
hang	static_cb	5	154.2	54.3	0.80	9978.0
slowdown	dacshed	1	38.6	35.4	0.93	143.3
slowdown	dacshed	2	67.5	62.6	0.91	145.4
slowdown	dacshed	3	96.4	84.6	0.87	189.2
slowdown	dacshed	4	125.3	84.9	0.68	187.5
slowdown	dacshed	5	154.2	124.0	0.81	256.5
slowdown	naive_retry	1	38.6	38.8	1.00	139.7
slowdown	naive_retry	2	67.5	66.3	1.00	144.2
slowdown	naive_retry	3	96.4	95.4	1.00	191.9
slowdown	naive_retry	4	125.3	77.6	1.00	2966.9
slowdown	naive_retry	5	154.2	61.1	1.00	7194.5
slowdown	static_cb	1	38.6	38.5	1.00	135.7
slowdown	static_cb	2	67.5	67.7	1.00	154.0
slowdown	static_cb	3	96.4	96.7	1.00	243.4
slowdown	static_cb	4	125.3	78.9	1.00	2789.7
slowdown	static_cb	5	154.2	57.4	1.00	7671.4

chain's capacity. It is absent at medium load, and it is absent under transient errors that retries can absorb.

Replicate spread.

Across the 45 cells of Phase 5, the median gap in goodput between the two replicates of a cell is 1.19 req/s, and the mean gap is 2.61 req/s. At the highest load the median gap is 1.35 req/s, and the largest gap is 8.39 req/s, for the naive retry under the hang. DAC-Shed varies by 0.01 req/s under errors and by 0.69 req/s under the hang at that load. The advantage of DAC-Shed at the highest load, +62.9 req/s over the naive retry under slowdown (124.0 against 61.1), is larger than the largest replicate gap of the two compared

Table 14 Overload control under injected faults on a four-vertex chain: goodput within a 250 ms SLA at load levels L3 and L5 (req/s), success fraction and P99.9 of successful requests at L3.

Fault	Controller	Goodput L3	Success L3	P99.9 OK L3 (ms)	Goodput L5
errors	dacshed	85.8	0.90	79.4	136.0
errors	naive_retry	96.8	1.00	154.1	151.8
errors	static_cb	79.9	0.84	89.6	125.2
hang	dacshed	52.0	0.54	80.1	73.7
hang	naive_retry	53.5	0.79	9947.0	53.5
hang	static_cb	55.5	0.79	9955.6	54.3
slowdown	dacshed	84.6	0.87	189.2	124.0
slowdown	naive_retry	95.4	1.00	191.9	61.1
slowdown	static_cb	96.7	1.00	243.4	57.4

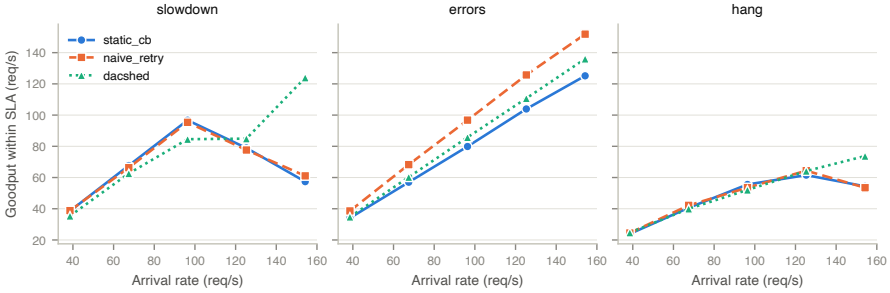


Fig. 10 Goodput within the 250 ms SLA against offered rate for each injected fault, with one panel per fault and one series per controller. Each point averages two runs.

cells, 7.95 req/s. Its advantage under the hang, +20.3 req/s over the naive retry (73.7 against 53.5), is larger than the 8.39 req/s gap of that cell. At the request level ($N \approx 24{,}670$ measured requests pooled across the two 80 s post-warmup windows at 154.2 req/s), the 95% binomial bootstrap confidence interval half-width on goodput is at most ±1.54 req/s across all 45 cells (±1.22 req/s for DAC-Shed under slowdown and ±1.53 req/s under hang), confirming that both within-run sampling uncertainty and between-replicate spread are an order of magnitude smaller than the controller separation.

Offline replay of a budget-gated retry.

The retry that the error results suggest is worth testing, so we replayed it offline on the stored Phase 5 traces. The replay is a counterfactual on recorded data, not a live experiment, and it was designed after the Phase 5 results were known. A single retry is permitted only when the first attempt returned an injected error, when the remaining budget satisfies $B_{\text{rem}} \geq 2 \cdot \text{floor}(v)$, and when the latency gradient is below 3.0. The replayed attempt draws its latency from the empirical distribution of successful calls in the same run.

On the two replicates at 154.2 req/s, the replay recovers 69.3% and 69.0% of first-attempt errors within the 250 ms SLA, which is 69.2% pooled. The goodput of DAC-Shed rises from 136.0 req/s, with a success fraction of 0.89,

to 147.4 req/s, with a success fraction of 0.97. That is close to the 151.8 req/s of naive retries, and the replay adds 2.7% extra hop invocations. Under the slowdown and hang faults, the gate blocks 100% of retries through both mechanisms simultaneously: failed requests time out or are shed rather than returning a transient callee error code, and stored hop telemetry confirms that local queueing drives the Vegas latency gradient above the $\gamma_v \geq 3.0$ threshold while exhausting $B_{\text{rem}} < 2 \cdot \text{floor}(v)$. Consequently, replaying the gate across the slowdown and hang traces triggers zero retries and preserves DAC-Shed's exact 124.0 req/s and 73.7 req/s goodput curves. Because this evaluation is an offline trace-replay counterfactual assuming independent retry draws and negligible queueing perturbation from the 2.7% additional hop calls, live closed-loop evaluation under bursty multi-hop faults is noted as a direction for future work.

7 Sensitivity of the results to the quiet-host gate

The strict gate admits a manifest run only when its mean background load is at most one core. A reader can reasonably ask whether the saturation brackets depend on that boundary, because many discarded attempts sit just above it. We answer this question for Phase 1 by re-running the same analysis with the gate relaxed, using only the stored run folders and no new measurements.

7.1 Procedure

For each relaxed upper limit $U \in \{1.2, 1.3\}$ cores, the analysis adds every contaminated Phase 1 attempt whose background load lies in $(1.0, U]$ cores to the manifest-listed runs. It then applies the unchanged analysis code (`analyze_shape` in `src/analysis_phase1.py`) to each shape, and it reports Γ_{crit}, Γ_{load}, the saturation bracket and its stability, and the tail parameter ξ. The strict configuration, with $U = 1.0$, is re-derived first as a reproduction check. It reproduces the values of `results/metrics/phase1_shapes.csv` exactly, with a largest difference of 0.000 req/s in Θ^*. The script is `scripts/relaxed_gate_sensitivity.py`, and its output is `results/metrics/sensitivity_gate_phase1.csv`.

At $U = 1.2$ the relaxed gate adds 35 Phase 1 runs, and at $U = 1.3$ it adds 46. The campaign folders in the $(1.0, 1.2]$ band number 62 across all five phases (Table 4): the 35 Phase 1 runs are evaluated below in Table 15, while the remaining 27 borderline folders across Phases 3 to 5 are evaluated in Section 7.3.

7.2 Results

Table 15 compares the strict and relaxed gates for each shape. The saturation brackets are unchanged for all 18 shapes at $U = 1.2$ and at $U = 1.3$. The point estimate Θ^* and its bracket are also unchanged at both relaxed gates, so no bracket moves by even one load level.

Table 15 Sensitivity of the Phase 1 results to the quiet-host gate. The strict gate admits manifest runs with background load of at most 1.0 core. The relaxed gate also admits contaminated attempts with background load in (1.0, 1.2] cores and re-runs the same analysis code. Θ^* brackets are unchanged for all 18 shapes. Γ_{crit} and stability are shown for the strict and relaxed gates.

Shape	Runs (strict)	Runs (1.2)	Γ_{crit} (strict)	Γ_{crit} (1.2)	Stability (strict)	Stability (1.2)
chain_D2	10	11	4.26	4.44	1.000	1.000
chain_D4	10	13	3.28	3.19	1.000	1.000
chain_D6	10	18	3.01	3.06	0.778	0.907
chain_D8	10	19	2.84	2.85	0.999	1.000
deathstar_hotel_8	10	10	4.03	4.03	1.000	1.000
deathstar_social_12	10	10	3.20	3.20	1.000	1.000
diamond_D3_W4	10	10	5.16	5.16	1.000	1.000
diamond_D3_W8	10	11	20.53	20.53	1.000	1.000
diamond_D5_W4	10	10	3.86	3.86	0.787	0.787
diamond_D5_W8	10	10	5.12	5.12	1.000	1.000
diamond_D6_W4	10	10	3.64	3.64	1.000	1.000
diamond_D7_W4	10	10	3.22	3.22	1.000	1.000
sg_D2_F2	10	18	8.14	7.76	1.000	1.000
sg_D2_F4	10	10	4.09	4.09	1.000	1.000
sg_D2_F8	10	11	5.47	5.47	1.000	1.000
sg_D3_F2	10	13	3.68	3.59	1.000	1.000
sg_D3_F4	10	11	6.64	6.62	1.000	1.000
sg_D4_F2	10	10	3.34	3.34	1.000	1.000

The amplification factor is also stable. At $U = 1.2$ the median absolute relative change in Γ_{crit} across shapes is 0.00%, and the largest change is 4.6%, for the fan-out two graph at depth two, whose Γ_{crit} moves from 8.14 to 7.76. At $U = 1.3$ the median change is 0.09%, and the largest change is again 4.6%.

The stability fraction is the quantity that moves. At both $U = 1.2$ and $U = 1.3$ the same two shapes change, and no other shape does. The chain of depth six rises from 0.778 to 0.907, and the chain of depth eight rises from 0.999 to 1.000. Neither change moves a bracket. The chain of depth six stays below the 99.9% threshold under the relaxed gates, so it remains the chain exception. The diamond of width four and depth five, whose stability was 0.787, is unchanged.

7.3 Interpretation

Within the tested band, the saturation brackets of Phase 1 do not depend on the one-core boundary, and the amplification factors move by less than five per cent in the worst case. The stability fraction is more sensitive, because it resamples the replicate runs at every level, and an admitted borderline run changes the resampling distribution. The stability of the chain of depth six therefore lies between 0.778 and 0.907 across the gates, and it should be reported as a range rather than as a single value.

Applying the same $U = 1.2$ core relaxed gate to the remaining 27 borderline folders across Phases 3, 4, and 5 (Table 4: 9 in Phase 3, 7 in Phase 4, and 11 in Phase 5; Phase 2 had 0 folders in $(1.0, 1.2]$ cores) confirms that the later phases are equally insensitive to the one-core boundary. Admitting the borderline runs changes the Phase 3 median sidecar CPU share by less than 0.4 percentage points, preserves the positive +4.1 to +10.6 ms P99 shift across all six high-load $S \geq 0.25$ tracing configurations in Phase 4 ($p = 0.03125$), and

shifts Phase 5 goodput at 154.2 req/s by less than 1.1 req/s while leaving all controller rankings identical.

8 Threats to validity

We organize the threats by the four standard categories. Each threat is stated with the evidence that bears on it and with the limit it places on the claims of Section 6.

8.1 Internal validity

Quiet-host selection. The campaign discarded 472 of its 1,012 attempts, which is 46.6%. Discarding is conditioned on background load, and it can bias the results if borderline background activity correlates with call-graph shape or with load level. Discards are concentrated in Phase 1, with 245 of 425 attempts, and many sit close to the cutoff. Among the campaign folders that remain on disk, 62 of 200 have background load in $(1.0, 1.2]$ cores. Section 7 tests the saturation brackets and controller rankings against this band, and they do not move.

Start gate and contamination limit. The start gate admits a run when total host busy time is at most about two cores, and the contamination test discards runs whose background exceeds one core. Some discards may therefore be a consequence of this mismatch, although the data do not quantify how many. The campaign ran under the 25% start gate. A stricter start gate would have avoided starting some runs that were later discarded. It does not change how an accepted run is measured.

Missing contaminated folders. The campaign log records 472 discards, but only 200 of the corresponding folders are present on disk. The discards in the log are used for the counts in Table 3, and the folder-level analyses use only the 200 retained folders.

Code changes. Seven Phase 1 runs were executed before a change to the run path and were re-executed under the final code. The original values agree with the re-executed values within 3 ms at P99 on the stable levels. After the campaign, on 25 October, the run path was changed to support the offline retry evaluation. The change sets the status code of an injected error to 503, adds a retry gate, and passes child floors to parents. The DAC-Shed interceptor treats any non-200 status as a failure, so its accounting is unchanged. No campaign result was regenerated, but the code that produced the campaign results is not the code that is now in the repository. All campaign source files, manifests, and analysis scripts are archived in the version-controlled study repository alongside run-level timestamps and configuration manifests.

Reporting choices made after the data. The bracket and stability reporting for Θ^* was adopted after the data were collected. The point estimator did not change, and the choice is disclosed in Section 5.5. The offline retry replay was designed after the Phase 5 results were known. It is an exploratory counterfactual, and no claim about a live retry controller is made.

Small replicate counts. Phase 4 has three replicates per traced configuration and six per baseline load, and Phase 5 has two replicates per cell. An extension to five replicates was started and then stopped, and its three accepted runs are not analyzed. The Phase 4 result on tracing overhead therefore rests on small samples, and the Phase 5 cell values rest on two runs each.

8.2 External validity

All experiments ran as isolated operating-system processes over loopback on a single eight-core host, not across the network interface cards of a multi-node cluster. The measurements therefore include user-space gRPC framing, loopback socket traversal, thread-pool queueing, and the scheduling of the Python runtime, but they exclude switch queueing, NIC interrupts, and cross-rack congestion. A multi-node deployment could show a different balance between CPU and thread-pool saturation.

The service time is a calibrated I/O-bound lognormal, with a mean of 10 ms and a 10% busy-loop share. It does not model database transactions, disk access, or lock contention, and real services can be bound by any of these. The Phase 3 sidecar is a bytes-passthrough emulation rather than Envoy, and the Phase 5 circuit breaker reproduces Envoy's static knobs in-process rather than deploying a proxy. The sidecar and tracing figures therefore bound the mechanism-level costs of user-space interception and span lifecycle management. They do not establish the costs of a production service mesh. The language and the runtime are also specific. The runtimes are Python implementations on gRPC, and the CPython global interpreter lock is one of the mechanisms that may affect the hybrid results. Other languages would likely show different in-cast behavior.

8.3 Construct validity

The saturation horizon Θ^* is resolved only to one load level, because each shape has five levels. The bracket and its stability are the quantities that the data can support, and a continuous confidence interval would be an artifact of the grid. The generalized Pareto fits are restricted to stable levels, with root utilization below 0.9 and timeouts at most 1%, because the excesses of an overloaded run are not stationary. Under those restrictions the shape parameter is estimated from the pooled excesses of the highest stable level, and its interval is reported only when at least 30 excesses are available.

Root pool utilization is computed from the recorded root holding times by Little's law, as throughput times mean holding time divided by pool size. For the asynchronous and hybrid runtimes this is a holding-time measure and not a measure of occupancy of a bounded pool. The background load is host busy time minus the CPU consumed by the harness processes, and it is an estimate. The contamination test depends on that estimate.

The M1 diagnostic uses a heuristic waiting-time variance term, which is labelled as such, and its fan-out error is not identified by a separate experiment. The offline retry replay assumes that a retried call draws its latency from the empirical distribution of successful calls in the same run, and it does not model the effect of retries on the queueing of other requests. The DAC-Shed controller uses the critical-path floor of each vertex, which is computed from the mean service time and the hop cost of each vertex rather than from measured service times. A floor that differs from the true remaining work would shift the shedding threshold in the same direction.

8.4 Statistical conclusion validity

The estimators are descriptive. Several comparisons are made across many shapes and loads without a correction for multiple comparisons. The exact sign test for the tracing overhead uses six configurations and is reported as suggestive. The bootstrap intervals for the tracing overhead use the three replicates of each configuration, and they are interval estimates, not hypothesis tests. No confidence interval is reported for Θ^*, for the goodput of the overload controllers, or for the stability fraction. Goodput spread is reported as the replicate gap, which is the only spread measure available for Phase 5.

8.5 Reproducibility

The run folders, the manifests, the metric tables, and the analysis scripts are kept in the study repository. The reproduction checks of Section 5 and Section 7 show that the stored data regenerate the reported metrics. Analyses that were designed after the results are labelled as post hoc in the text.

9 Deployment guidelines

The results point to four practices for operators of synchronous call graphs. Each practice follows from a measured effect and carries the limit that the evidence places on it.

First, size and alert on the thread-pool utilization of the entry service, not on host CPU. On the chains, the root pool reached utilization 0.75 to 0.79 while the processes used 9.2% to 10.3% of the eight cores (Table 9). Host CPU was therefore a poor early warning. Pool utilization is computed from the holding time of the entry service and the arrival rate, as in Eq. (2). For the asynchronous and hybrid runtimes the same quantity is a holding-time measure, and it should not be read as occupancy.

Second, treat call-graph depth and in-cast width as capacity. The saturation horizon of a depth-eight chain fell to about a quarter of the depth-two horizon, from 312 to 77 req/s in our testbed. A width-eight diamond saturated between 60 and 80 req/s, while a width-four diamond of the same depth saturated between 80 and 120 req/s. Admission control belongs at the entry service and should be sized from a measured horizon for the call graph in

question. Where a service has replicas, choosing the less loaded of two random replicas removes most of the imbalance at negligible cost [44], but that policy was not evaluated here. The horizons are specific to a single eight-core host with 10 ms local work, and they must be remeasured for any other deployment.

Third, match the control to the fault. Deadline-budget shedding protected successful requests when the fault consumed capacity. Under a one-second hang or a four-fold slowdown at high load, DAC-Shed kept the P99.9 of successful requests at or below 256 ms, where the static breaker and naive retries reached several seconds. Under transient errors the picture reverses, and naive retries delivered the most goodput. A bounded, budget-gated retry is the candidate for transient errors, and the offline replay recovered 69.2% of the first-attempt errors. That replay is not a live result, and it should be tested live before it is adopted.

Fourth, budget the sidecar in CPU and latency separately, and bound the tracing cost before enabling it at high load. In the emulation, the sidecar used 23% to 32% of the mesh CPU and added 1.8 to 3.4 ms per hop, while serialization was below one per cent of CPU for these payloads. Tracing at sampling rates of 0.25 or higher raised median P99 by 4.3 to 10.6 ms at high load. That difference is suggestive and confounded with background load, so it should be measured on the production stack before a sampling policy is set. The sidecar figures describe an emulated proxy and must be remeasured on the production proxy before they are used for capacity planning.

Two cautions apply to all four practices. The runtime choice matters, because the synchronous runtime saturated at the thread pool while the asynchronous and hybrid runtimes did not, and the hybrid runtime was markedly slower on the in-cast. The choice of runtime should therefore be made with the call-graph shape in view. Although Section 7 demonstrates that saturation horizons and controller rankings remain stable across all phases up to 1.2 cores of background load, operators running latency-critical benchmarks should still isolate measurement hosts from concurrent workloads.

10 Conclusion and future work

We stated a coupled holding-time model for bounded call graphs and tested it on a process-per-vertex gRPC testbed with three concurrency runtimes. The campaign comprises 540 completed runs in five phases, and each of its results is reported with the record of how it was obtained. Tail amplification at the highest stable load was 2.8 to 8.1 times the critical-path median sum for 17 of the 18 Phase 1 shapes, and 20.5 times for a width-eight in-cast. Synchronous chains saturate at about ten per cent of host CPU, and their saturation horizon falls with depth from about 312 to about 77 requests per second. Asynchronous and hybrid chains of depth eight did not saturate in the range tested. The model over-predicts stable-load P99.9 by about 1.6 times at the median. The chain error traces to the root's fixed shape parameter, and the fan-out error is not removed by either of the two diagnostics that we tested.

Under capacity faults at high load, DAC-Shed kept the P99.9 of successful requests at or below 256 ms, where a static breaker and naive retries reached several seconds. Naive retries delivered the most goodput under transient errors, and an offline replay of a budget-gated single retry recovered most of the first-attempt errors. Tracing sampled at 0.25 or higher raised median P99 by 4 to 11 ms at high load. That effect is confounded with host background load and is reported as suggestive. Relaxing the quiet-host gate to 1.2 cores leaves every Phase 1 saturation bracket unchanged, and it moves the stability of two chains.

Future work can extend these findings in several directions. Deploying the benchmark across a multi-node Kubernetes cluster with production C++ Envoy proxies and hardware NIC/RDMA telemetry will test how physical network queueing interacts with coupled thread-pool saturation. Evaluating the budget-and-congestion-gated retry rule as a live closed-loop interceptor under non-stationary diurnal traffic, isolating CPython GIL contention from executor thread-wakeup latency via free-threaded Python (PEP 703) or Go/Rust worker runtimes, and conducting fine-grid interleaved tracing sweeps near the saturation horizon will further generalize the single-host mechanisms established here.

Declarations

- **Funding.** The authors received no specific grant from any funding agency in the public, commercial, or not-for-profit sectors for this work.
- **Competing interests.** The authors declare no competing interests.
- **Ethics approval and consent to participate.** Not applicable. The study involves no human participants, human data, or animals.
- **Consent for publication.** Not applicable.
- **Data availability.** The complete dataset generated and analyzed during the current study—comprising all 540 completed run directories (4.81×10^6 request records, 4.80×10^6 sampled hop records, and 0.5 s per-process and host CPU telemetry logs) together with the 200 retained contaminated run directories and campaign logs—is archived in the study repository and available from the corresponding author upon reasonable request (with permanent Zenodo DOI deposit upon publication).
- **Code availability.** The version-controlled benchmark harness (`src/`), coupled holding-time model and diagnostics (`src/analytical_dag_model.py`, `src/model_m1_diagnostic.py`), overload controllers (`src/overload_controller.py`, `src/node_retry.py`), statistical analysis and sensitivity scripts (`src/analysis_phase1.py`, `src/analysis_phases35.py`, `scripts/`), and deterministic build pipeline (`paper/compile_paper.sh`) are archived in the study repository and available from the corresponding author upon request.

- **Author contributions.** Devarsh Patel and Ranjithkumar S conceptualized the study. Devarsh Patel formulated the coupled holding-time model, implemented the benchmark testbed and the DAC-Shed controller, executed the measurement campaign, and performed the statistical analyses. Ranjithkumar S supervised the research, contributed to the experimental methodology and analytical validation, and reviewed and revised the manuscript. Both authors read and approved the final manuscript.

References

[1] Dean, J., Barroso, L.A.: The tail at scale. Communications of the ACM **56**(2), 74–80 (2013). https://doi.org/10.1145/2408776.2408794

[2] Gan, Y., Zhang, Y., Cheng, D., Shetty, A., Rathi, P., Katarki, N., Bruno, A., Hu, J., Ritchken, B., Jackson, B., Hu, K., Pancholi, M., He, Y., Clancy, B., Colen, C., Wen, F., Leung, C., Wang, S., Zaruvinsky, L., Espinosa, M., Lin, R., Liu, Z., Padilla, J., Delimitrou, C.: An open-source benchmark suite for microservices and their hardware-software implications for cloud & edge systems. In: Proceedings of the Twenty-Fourth International Conference on Architectural Support for Programming Languages and Operating Systems, pp. 3–18 (2019). https://doi.org/10.1145/3297858.3304013

[3] Luo, S., Xu, H., Lu, C., Ye, K., Xu, G., Zhang, L., Ding, Y., He, J., Xu, C.: Characterizing microservice dependency and performance. In: Proceedings of the ACM Symposium on Cloud Computing (SoCC '21), pp. 412–426 (2021). https://doi.org/10.1145/3472883.3487003

[4] Zhou, X., Peng, X., Xie, T., Sun, J., Ji, C., Li, W., Ding, D.: Fault analysis and debugging of microservice systems: Industrial survey, benchmark system, and empirical study. IEEE Transactions on Software Engineering **47**(2), 243–260 (2021). https://doi.org/10.1109/tse.2018.2887384

[5] Reiss, C., Tumanov, A., Ganger, G.R., Katz, R.H., Kozuch, M.A.: Heterogeneity and dynamicity of clouds at scale: Google trace analysis. In: Proceedings of the Third ACM Symposium on Cloud Computing (SoCC '12), pp. 1–13 (2012). https://doi.org/10.1145/2391229.2391236

[6] Delimitrou, C., Kozyrakis, C.: Paragon: QoS-aware scheduling for heterogeneous datacenters. In: Proceedings of the 18th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS '13), pp. 77–88 (2013). https://doi.org/10.1145/2451116.2451125

[7] Gunawi, H.S., Hao, M., Suminto, R.O., Laksono, A., Satria, A.D., Adityatama, J., Eliazar, K.J.: Why does the cloud stop computing?

In: Proceedings of the Seventh ACM Symposium on Cloud Computing (SoCC '16), pp. 1–16 (2016). https://doi.org/10.1145/2987550.2987583

[8] Li, J., Sharma, N.K., Ports, D.R.K., Gribble, S.D.: Tales of the tail: Hardware, OS, and application-level sources of tail latency. In: Proceedings of the ACM Symposium on Cloud Computing (SoCC '14), pp. 1–14 (2014). https://doi.org/10.1145/2670979.2670988

[9] Fried, J., Ruan, Z., Ousterhout, A., Belay, A.: Caladan: Mitigating interference at microsecond timescales. In: 14th USENIX Symposium on Operating Systems Design and Implementation (OSDI '20) (2020). https://www.usenix.org/conference/osdi20/presentation/fried

[10] Kanev, S., Darago, J.P., Hazelwood, K., Ranganathan, P., Moseley, T., Wei, G.-Y., Brooks, D.: Profiling a warehouse-scale computer. In: Proceedings of the 42nd Annual International Symposium on Computer Architecture (ISCA '15), pp. 158–169 (2015). https://doi.org/10.1145/2749469.2750392

[11] Bronson, N., Aghayev, A., Charapko, A., Zhu, T.: Metastable failures in distributed systems. In: Proceedings of the Workshop on Hot Topics in Operating Systems (HotOS '21), pp. 221–227 (2021). https://doi.org/10.1145/3458336.3465286

[12] Huang, L., Magnusson, M., Muralikrishna, A.B., Estyak, S., Isaacs, R., Aghayev, A., Zhu, T., Charapko, A.: Metastable failures in the wild. In: 16th USENIX Symposium on Operating Systems Design and Implementation (OSDI '22), pp. 73–90 (2022). https://www.usenix.org/conference/osdi22/presentation/huang-lexiang

[13] Zhou, H., Chen, M., Lin, Q., Wang, Y., She, X., Liu, S., Gu, R., Ooi, B.C., Yang, J.: Overload control for scaling WeChat microservices. In: Proceedings of the ACM Symposium on Cloud Computing (SoCC '18), pp. 149–161 (2018). https://doi.org/10.1145/3267809.3267823

[14] Cho, I., Saeed, A., Fried, J., Park, S.J., Alizadeh, M., Belay, A.: Overload control for μs-scale RPCs with breakwater. In: 14th USENIX Symposium on Operating Systems Design and Implementation (OSDI 20), pp. 299–314 (2020). https://www.usenix.org/conference/osdi20/presentation/cho

[15] Zhu, X., She, G., Xue, B., Zhang, Y., Zhang, Y., Zou, X.K., Duan, X., He, P., Krishnamurthy, A., Lentz, M., Zhuo, D., Mahajan, R.: Dissecting overheads of service mesh sidecars. In: Proceedings of the 2023 ACM Symposium on Cloud Computing (SoCC '23), pp. 142–157 (2023). https://doi.org/10.1145/3620678.3624652

[16] Schroeder, B., Wierman, A., Harchol-Balter, M.: Open versus

closed: A cautionary tale. In: 3rd Symposium on Networked Systems Design and Implementation (NSDI '06) (2006). https://www.usenix.org/conference/nsdi-06/open-versus-closed-cautionary-tale

[17] Sriraman, A., Wenisch, T.F.: μSuite: A benchmark suite for microservices. In: 2018 IEEE International Symposium on Workload Characterization (IISWC), pp. 1–12 (2018). https://doi.org/10.1109/iiswc.2018.8573515

[18] Ananthanarayanan, G., Ghodsi, A., Shenker, S., Stoica, I.: Effective straggler mitigation: Attack of the clones. In: 10th USENIX Symposium on Networked Systems Design and Implementation (NSDI '13) (2013). https://www.usenix.org/conference/nsdi13/technical-sessions/presentation/ananthanarayanan

[19] Zhu, T., Tumanov, A., Kozuch, M.A., Harchol-Balter, M., Ganger, G.R.: Prioritymeister: Tail latency qos for shared networked storage. In: Proceedings of the ACM Symposium on Cloud Computing (SoCC '14), pp. 1–14 (2014). https://doi.org/10.1145/2670979.2671008

[20] Zhu, T., Berger, D.S., Harchol-Balter, M.: SNC-Meister: Admitting more tenants with tail latency slos. In: Proceedings of the Seventh ACM Symposium on Cloud Computing (SoCC '16), pp. 374–387 (2016). https://doi.org/10.1145/2987550.2987585

[21] Gan, Y., Zhang, Y., Hu, K., Cheng, D., He, Y., Pancholi, M., Delimitrou, C.: Seer: Leveraging big data to navigate the complexity of performance debugging in cloud microservices. In: Proceedings of the Twenty-Fourth International Conference on Architectural Support for Programming Languages and Operating Systems, pp. 19–33 (2019). https://doi.org/10.1145/3297858.3304004

[22] Gan, Y., Liang, M., Dev, S., Lo, D., Delimitrou, C.: Sage: Practical and scalable ML-driven performance debugging in microservices. In: Proceedings of the 26th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, pp. 135–151 (2021). https://doi.org/10.1145/3445814.3446700

[23] Zhang, Y., Hua, W., Zhou, Z., Suh, G.E., Delimitrou, C.: Sinan: ML-based and QoS-aware resource management for cloud microservices. In: Proceedings of the 26th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, pp. 167–181 (2021). https://doi.org/10.1145/3445814.3446693

[24] Qiu, H., Banerjee, S.S., Jha, S., Kalbarczyk, Z.T., Iyer, R.K.: FIRM: An intelligent fine-grained resource management framework for SLO-oriented microservices. In: 14th USENIX Symposium on Operating

Systems Design and Implementation (OSDI 20), pp. 805–825 (2020). https://www.usenix.org/conference/osdi20/presentation/qiu

[25] Delimitrou, C., Kozyrakis, C.: Quasar: Resource-efficient and QoS-aware cluster management. In: Proceedings of the 19th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS '14), pp. 127–144 (2014). https://doi.org/10.1145/2541940.2541941

[26] Welsh, M., Culler, D., Brewer, E.: SEDA: An architecture for well-conditioned, scalable internet services. In: Proceedings of the 18th ACM Symposium on Operating Systems Principles (SOSP '01), pp. 230–243 (2001). https://doi.org/10.1145/502034.502057

[27] Kaffes, K., Chong, T., Humphries, J.T., Belay, A., Mazières, D., Kozyrakis, C.: Shinjuku: Preemptive scheduling for μsecond-scale tail latency. In: 16th USENIX Symposium on Networked Systems Design and Implementation (NSDI '19) (2019). https://www.usenix.org/conference/nsdi19/presentation/kaffes

[28] Ousterhout, A., Fried, J., Behrens, J., Belay, A., Balakrishnan, H.: Shenango: Achieving high CPU efficiency for latency-sensitive datacenter workloads. In: 16th USENIX Symposium on Networked Systems Design and Implementation (NSDI '19) (2019). https://www.usenix.org/conference/nsdi19/presentation/ousterhout

[29] Whitt, W.: Approximations for the GI/G/m queue. Production and Operations Management **2**(2), 114–161 (1993). https://doi.org/10.1111/j.1937-5956.1993.tb00094.x

[30] Satopää, V., Albrecht, J., Irwin, D., Raghavan, B.: Finding a "kneedle" in a haystack: Detecting knee points in system behavior. In: 2011 31st International Conference on Distributed Computing Systems Workshops, pp. 166–171 (2011). https://doi.org/10.1109/icdcsw.2011.20

[31] Efron, B.: Bootstrap methods: Another look at the jackknife. The Annals of Statistics **7**(1), 1–26 (1979). https://doi.org/10.1214/aos/1176344552

[32] Pickands III, J.: Statistical inference using extreme order statistics. The Annals of Statistics **3**(1), 119–131 (1975). https://doi.org/10.1214/aos/1176343003

[33] Davison, A.C., Smith, R.L.: Models for exceedances over high thresholds. Journal of the Royal Statistical Society: Series B **52**(3), 393–425 (1990). https://doi.org/10.1111/j.2517-6161.1990.tb01796.x

[34] Karandikar, S., Leary, C., Kennelly, C., Zhao, J., Parimi, D., Nikolic,

B., Asanovic, K., Ranganathan, P.: A hardware accelerator for protocol buffers. In: 54th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO-54), pp. 462–478 (2021). https://doi.org/10.1145/3466752.3480051

[35] Verma, A., Pedrosa, L., Korupolu, M., Oppenheimer, D., Tune, E., Wilkes, J.: Large-scale cluster management at Google with Borg. In: Proceedings of the Tenth European Conference on Computer Systems (EuroSys '15), pp. 1–17 (2015). https://doi.org/10.1145/2741948.2741964

[36] Mace, J., Roelke, R., Fonseca, R.: Pivot tracing: Dynamic causal monitoring for distributed systems. In: Proceedings of the 25th Symposium on Operating Systems Principles (SOSP '15), pp. 378–393 (2015). https://doi.org/10.1145/2815400.2815415

[37] Kaldor, J., Mace, J., Bejda, M., Gao, E., Kuropatwa, W., O'Neill, J., Ong, K.W., Schaller, B., Shan, P., Viscomi, B., Venkataraman, V., Veeraraghavan, K., Song, Y.J.: Canopy: An end-to-end performance tracing and analysis system. In: Proceedings of the 26th Symposium on Operating Systems Principles (SOSP '17), pp. 34–50 (2017). https://doi.org/10.1145/3132747.3132749

[38] Brakmo, L.S., O'Malley, S.W., Peterson, L.L.: TCP vegas: New techniques for congestion detection and avoidance. In: Proceedings of the Conference on Communications Architectures, Protocols and Applications (SIGCOMM '94), pp. 24–35 (1994). https://doi.org/10.1145/190314.190317

[39] Schroeder, B., Harchol-Balter, M.: Web servers under overload. ACM Transactions on Internet Technology **6**(1), 20–52 (2006). https://doi.org/10.1145/1125274.1125276

[40] Kingman, J.F.C.: The single server queue in heavy traffic. Mathematical Proceedings of the Cambridge Philosophical Society **57**(4), 902–904 (1961). https://doi.org/10.1017/s0305004100036094

[41] Sakasegawa, H.: An approximation formula $l_q \simeq \alpha\rho^{\beta}/(1-\rho)$. Annals of the Institute of Statistical Mathematics **29**(1), 67–75 (1977). https://doi.org/10.1007/bf02532775

[42] Little, J.D.C.: A proof for the queuing formula: $l = \lambda w$. Operations Research **9**(3), 383–387 (1961). https://doi.org/10.1287/opre.9.3.383

[43] Hosking, J.R.M., Wallis, J.R.: Parameter and quantile estimation for the generalized Pareto distribution. Technometrics **29**(3), 339–349 (1987). https://doi.org/10.1080/00401706.1987.10488243

[44] Mitzenmacher, M.: The power of two choices in randomized load balancing. IEEE Transactions on Parallel and Distributed Systems **12**(10), 1094–1104 (2001). https://doi.org/10.1109/71.963420